

# 1. INTRODUCTION À JAVA

## QU'EST-CE QUE JAVA ?

Java est un langage de programmation et une plate-forme informatique que Sun Microsystems a lancé pour la première fois en 1995. Il s'agit d'un langage orienté objet, basé sur des classes, conçu pour être portable, ce qui signifie que le code de Java peut fonctionner sur une grande variété de matériel et de systèmes d'exploitation. Java est largement utilisé pour le développement d'applications d'entreprise, d'applications mobiles, de jeux vidéo et d'autres types de logiciels. Il est connu pour sa philosophie « écrire une fois, exécuter n'importe où », car le code Java peut être compilé pour s'exécuter sur n'importe quelle plate-forme prenant en charge une machine virtuelle Java (JVM). En outre, Java dispose d'un écosystème important et actif, avec une multitude de bibliothèques et de frameworks pour les développeurs<sup>1</sup>.

Java est également connu pour ses fonctions de sécurité, car il est conçu pour être sécurisé dès le départ grâce à des fonctions telles que la gestion de la mémoire et le traitement automatique des exceptions. La fonction de gestion de la mémoire de Java, appelée « garbage collector », libère automatiquement la mémoire qui n'est plus utilisée, ce qui réduit la probabilité de fuites de mémoire. La fonction de traitement automatique des exceptions permet aux développeurs d'écrire plus facilement du code robuste capable de gérer des erreurs inattendues.

Java est un langage de programmation puissant et polyvalent, idéal pour le développement d'applications modernes. Grâce à son orientation objet, sa portabilité et ses caractéristiques de sécurité, il est largement utilisé dans divers domaines, allant du développement d'applications de bureau à la création d'applications web et mobiles.

### 1.1.1. Historique et caractéristiques principales

Né au début des années **1990** sous le nom de Project Oak, Java a été conçu à l'origine pour devenir un langage de programmation de niche destiné à être utilisé dans l'industrie de la télévision numérique par câble pour programmer des appareils numériques tels que

---

<sup>1</sup> [Qu'est-ce que Java ? Définition, signification et caractéristiques | AppMaster](#)

les décodeurs et les téléviseurs intelligents. Il a fallu des années et de multiples changements pour amener Java là où il est maintenant. Comme le dit le proverbe, revenez à la racine et vous trouverez le sens.

Java est né en juin 1991 sous la forme d'un projet appelé « Oak » sous le développement d'une petite équipe d'ingénieurs travaillant pour Sun Microsystems. Ils se sont appelés l'équipe verte : **James Gosling, Mike Sheridan et Patrick Naughton**. Et le mot « Chêne » a été choisi pour nommer la nouvelle technologie car le chêne est un symbole de force et de durabilité.

**James Gosling** était à la tête du projet, et son objectif initial était de créer un langage de programmation orienté objet qui pourrait implémenter une machine virtuelle et serait plus simple et plus universel que C/C++, mais en même temps aurait une syntaxe similaire à C/C++ pour le rendre facile à apprendre et à utiliser par les programmeurs actuels qui connaissent bien la notation C, c'est au début de **1996** (23 janvier 1996) que Sun Microsystems a publié la première implémentation publique de Java 1.0.

En résumé :

**1991** : Le projet Java a été initié par James Gosling et une équipe chez Sun Microsystems dans le but de créer un langage de programmation pour les appareils électroniques (télévision interactive, etc.).

**1996** : La première version publique de Java (Java 1.0) a été lancée. Java a été conçu avec l'idée que « Écrire une fois, exécuter partout » (WORA), ce qui signifie que les programmes Java peuvent être exécutés sur n'importe quelle plateforme qui dispose d'une JVM (Java Virtual Machine).

**1999** : Java 2 a été introduit, apportant des améliorations significatives, notamment la bibliothèque Swing pour les interfaces graphiques et les Collections Framework.

**2004** : Java 5 a été lancé, introduisant des fonctionnalités importantes comme les génériques, les annotations, et les énumérations.

**2010** : Oracle a acquis Sun Microsystems, intégrant Java dans sa suite de produits.

**2017** : Java suit un cycle de publication régulier, avec des mises à jour tous les six mois, permettant d'apporter de nouvelles fonctionnalités de manière continue.

Depuis septembre 2021, la dernière version est Java 16 ou JDK 16 libéré le 16 mars 2021. Java 16 a apporté 17 nouvelles améliorations à la plate-forme qui amélioreront encore la productivité des développeurs. « La puissance de la cadence de publication de six mois était pleinement visible avec la dernière version.

### 1.1.2. - Plateforme Java (JDK, JRE, JVM)

Le **Java Development Kit (JDK)** est un ensemble d'outils nécessaires pour développer des applications Java. Il comprend :

**JDK (Java Development Kit) :**

Le JDK est un ensemble d'outils nécessaires pour développer des programmes Java. Il inclut le compilateur Java (**javac**), les bibliothèques Java, des outils de développement et des exemples de code. Le JDK est essentiel pour les développeurs qui écrivent et compilent du code Java. **javac** pour transformer le code source Java en bytecode. Il a aussi l'outil de développement comme **javadoc** pour générer de la documentation.

**JRE (Java Runtime Environment) :**

- Le JRE est l'environnement d'exécution qui permet d'exécuter des applications Java. Il contient la JVM, ainsi que les bibliothèques et les fichiers nécessaires pour faire fonctionner les applications Java. Les utilisateurs finaux qui souhaitent exécuter des applications Java n'ont besoin que du JRE.

**JVM (Java Virtual Machine) :**

- La JVM est la machine virtuelle qui exécute le bytecode Java. Elle agit comme un interpréteur, traduisant le bytecode en instructions spécifiques à la plateforme sous-jacente. La JVM offre également des fonctionnalités de sécurité et de gestion de la mémoire. Grâce à la JVM, le même bytecode peut être exécuté sur n'importe quelle machine, quelle que soit sa configuration matérielle ou son système d'exploitation.

### 1.1.3. Installation et Configuration

L'installation et la configuration du JDK ainsi que des IDE sont des étapes essentielles pour commencer à développer en Java. Suivre ces étapes vous permettra de créer un environnement de développement fonctionnel et adapté à vos besoins.

#### Installation du JDK

Le Java Development Kit (JDK) est un ensemble d'outils essentiels pour le développement d'applications Java. Voici les étapes pour l'installer :

#### Étape 1 : Téléchargement du JDK

1. Accéder au site officiel :

Allez sur le site d'[Oracle](https://www.oracle.com) ou sur le site d'[OpenJDK](https://openjdk.org) pour télécharger la version souhaitée du JDK.

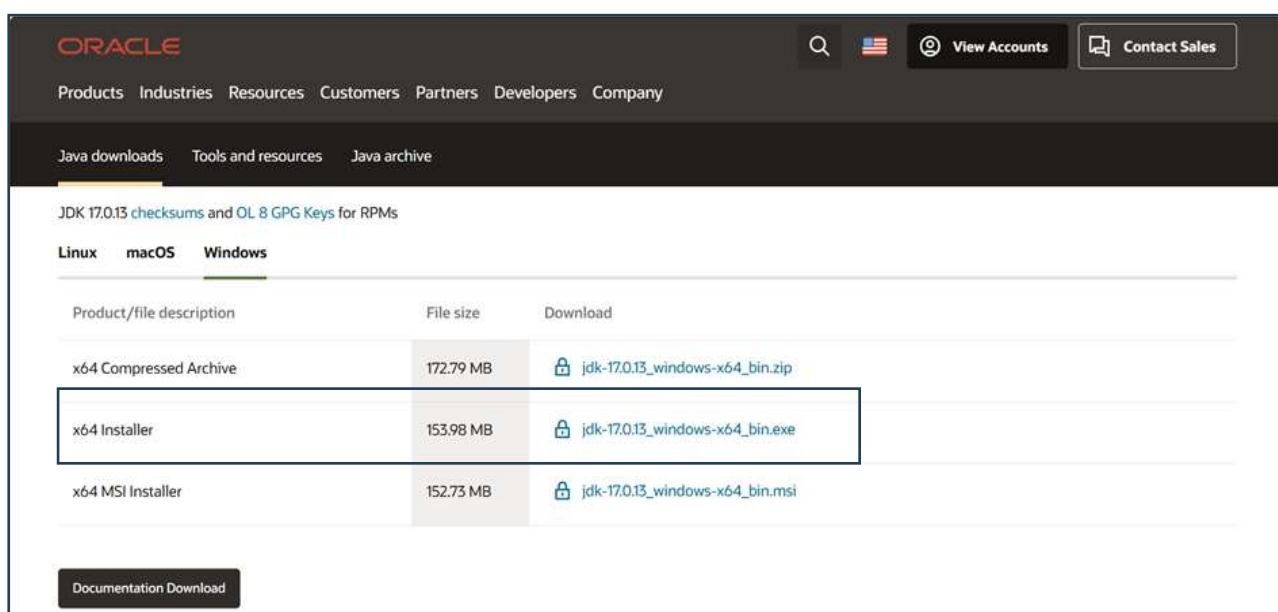


Figure 1: <https://www.oracle.com/java/technologies/downloads/#java17-windows>

2. Choisir la version :
  - Sélectionnez la version du JDK que vous souhaitez installer (par exemple, JDK 17 ou JDK 21).
3. Sélectionner le système d'exploitation :

- Choisissez le bon package pour votre système d'exploitation (Windows, macOS, Linux).

## Étape 2 : Installation du JDK

1. Exécuter le programme d'installation :
  - Téléchargez le fichier d'installation et exécutez-le.

Cliquez sur pour le lien de téléchargement ci-haut ;

Acceptez le contrat de téléchargement



Après avoir eu un fichier exécutable, exécutez-le ;



Figure 2: Fichier .exe téléchargé

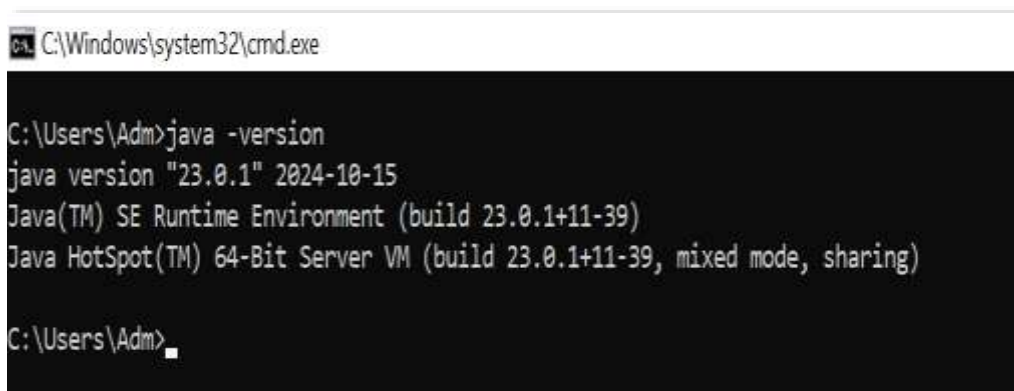
2. Suivre les instructions à l'écran :

Acceptez le contrat de licence et choisissez l'emplacement d'installation (vous pouvez laisser le chemin par défaut).

3. Vérification de l'installation :

Ouvrez une invite de commande (Windows) ou un terminal (MacOs/Linux) ;

Tapez la commande suivante « *java -version* » pour vérifier l'installation :



```
C:\Windows\system32\cmd.exe

C:\Users\Adm>java -version
java version "23.0.1" 2024-10-15
Java(TM) SE Runtime Environment (build 23.0.1+11-39)
Java HotSpot(TM) 64-Bit Server VM (build 23.0.1+11-39, mixed mode, sharing)

C:\Users\Adm>
```

*Figure 3:version de JDK*

Vous devriez voir des informations sur la version installée du JDK.

## **Configuration de l'environnement de développement**

Il existe plusieurs environnements de développement intégrés (EDI) tels qu'Eclipse, IntelliJ IDEA et NetBeans. Dans le cadre de cette unité d'enseignement, nous allons utiliser NetBeans.

### **NetBeans**

NetBeans est un environnement de développement intégré (EDI), placé en open source par Sun en juin 2000 sous licence CDDL (Common Development and Distribution License) et GPLv2. En plus de Java, NetBeans permet la prise en charge native de divers langages tels le C, le C++, le JavaScript, le XML, le Groovy, le PHP et le HTML, ou d'autres (dont Python et Ruby) par l'ajout de greffons. Il offre toutes les facilités d'un IDE moderne (éditeur avec coloration syntaxique, projets multi-langage, refactoring, éditeur graphique d'interfaces et de pages Web).

Compilé en Java, NetBeans est disponible sous Windows, Linux, Solaris (sur x86 et SPARC), Mac OS X ou sous une version indépendante des systèmes d'exploitation (requérant une machine virtuelle Java). Un environnement Java Development Kit JDK est requis pour les développements en Java.

NetBeans constitue par ailleurs une plateforme qui permet le développement d'applications spécifiques (bibliothèque Swing (Java)). L'IDE NetBeans s'appuie sur cette plateforme.

En 1997, NetBeans naît de Xelfi, un projet d'étudiants dirigé par la Faculté de mathématiques et de physique de l'Université Charles de Prague. Plus tard, une société se forme autour du projet et édite des versions commerciales de l'EDI NetBeans, jusqu'à ce qu'il soit acheté par Sun en 1999. Sun place le projet sous double licence CDDL et GPL v2 en juin de l'année suivante.

En septembre 2016, Oracle, qui a racheté Sun, propose de céder le projet à la fondation Apache Software Foundation, ce qui est accepté en octobre 2016.

### 1. Compatibilité entre JDK et NetBeans

| Version de JDK | Version de NetBeans          | Remarques                                   |
|----------------|------------------------------|---------------------------------------------|
| <b>JDK 8</b>   | <b>NetBeans 8.0 - 8.2</b>    | Support complet pour Java SE 8              |
| <b>JDK 9</b>   | NetBeans 9.0                 | Support limité ; nécessite des mises à jour |
| <b>JDK 10</b>  | NetBeans 10.0                | Support complet pour Java SE 10             |
| <b>JDK 11</b>  | NetBeans 11.0                | Support complet pour Java SE 11             |
| <b>JDK 12</b>  | NetBeans 11.0 et ultérieures | Support partiel                             |
| <b>JDK 13</b>  | NetBeans 11.0 et ultérieures | Support partiel                             |
| <b>JDK 14</b>  | NetBeans 11.0 et ultérieures | Support partiel                             |
| <b>JDK 15</b>  | NetBeans 11.3 et ultérieures | Support partiel                             |
| <b>JDK 16</b>  | NetBeans 12.0 et ultérieures | Support complet pour Java SE 16             |
| <b>JDK 17</b>  | NetBeans 12.0 et ultérieures | Support complet pour Java SE 17             |
| <b>JDK 18</b>  | NetBeans 12.0 et ultérieures | Support partiel, mise à jour recommandée    |
| <b>JDK 19</b>  | NetBeans 12.6 et ultérieures | Support partiel, mise à jour recommandée    |
| <b>JDK 20</b>  | NetBeans 12.6 et ultérieures | Support partiel, mise à jour recommandée    |

*Remarques Importantes*1.

**Versions LTS** : Les versions JDK 8, 11, et 17 sont des versions Long Term Support (LTS) et bénéficient d'un support prolongé. Il est donc recommandé de les utiliser pour des projets à long terme.

**2. Mises à Jour** : Assurez-vous d'utiliser la version la plus récente de NetBeans pour bénéficier des dernières améliorations et corrections de bugs.

3. **Compatibilité avec d'autres IDE** : Si vous utilisez d'autres IDE comme IntelliJ IDEA ou Eclipse, vérifiez également leur compatibilité avec les versions de JDK.
4. **Documentation Officielle** : Pour des détails supplémentaires et des mises à jour sur les versions, consultez la documentation officielle de [NetBeans](https://netbeans.apache.org) et [Oracle JDK](https://www.oracle.com/technetwork/java/javase-downloads-1387621.html).

Assurez-vous de choisir des versions compatibles pour garantir un fonctionnement optimal de votre IDE et de vos projets Java.

## 2. Téléchargement :

- Allez sur le site de [NetBeans](https://netbeans.apache.org) et téléchargez la dernière version.

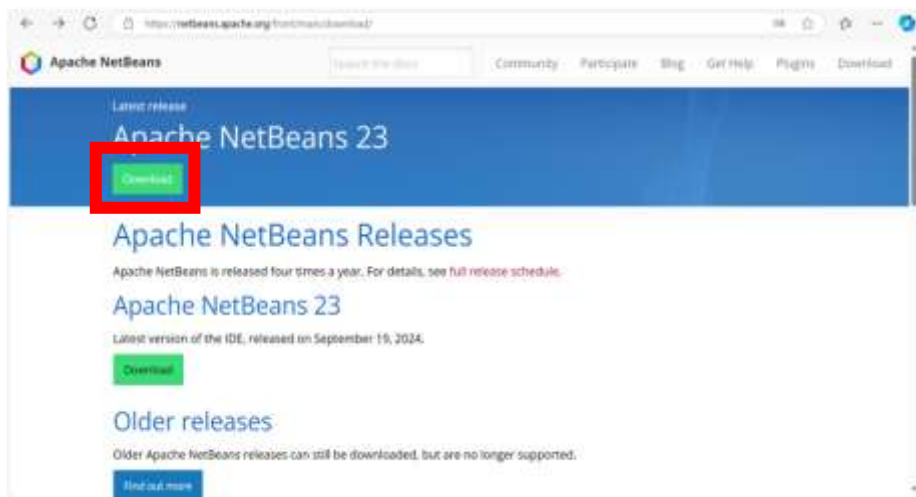
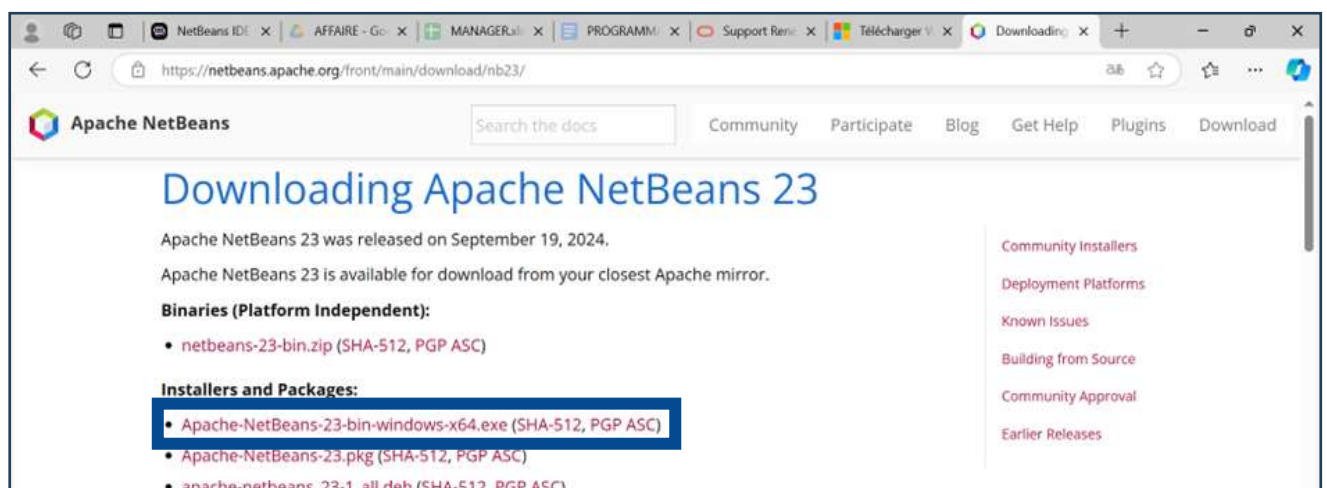
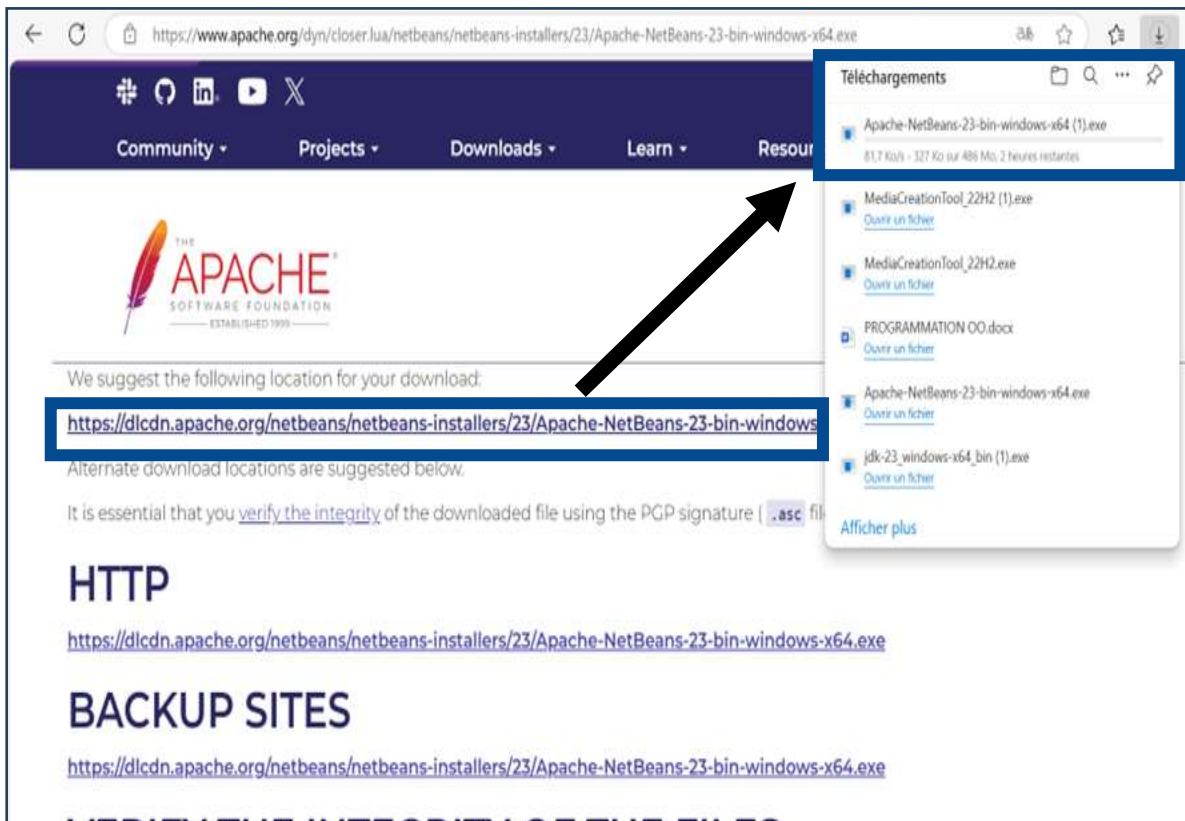


Figure 4: Site NetBeans





### 3. Exécution du programme d'installation :

- Suivez les instructions à l'écran pour installer NetBeans.

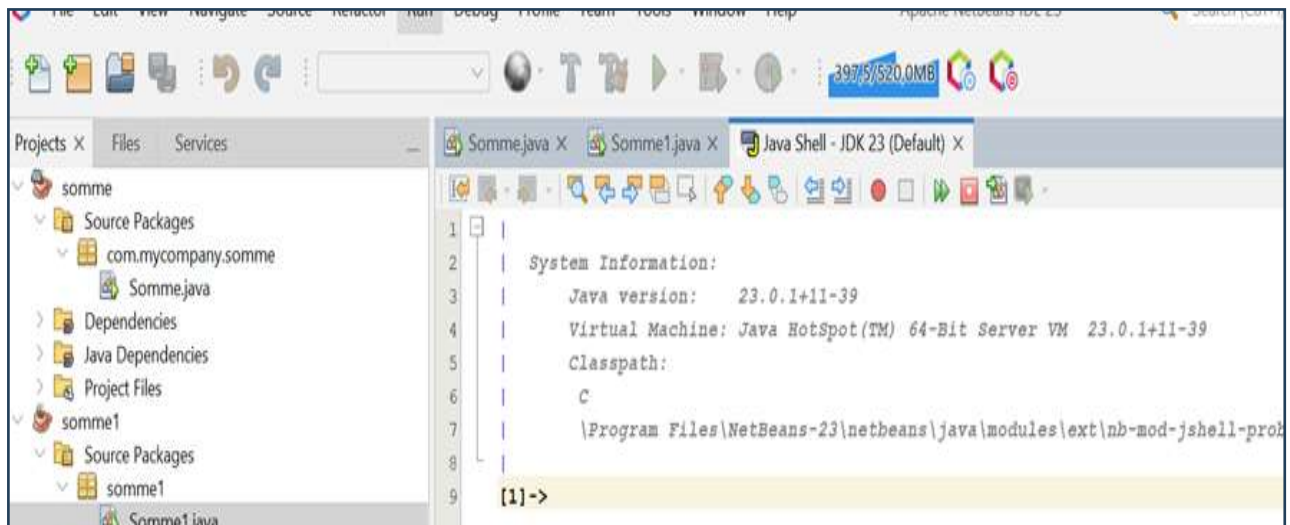
Configuration :



Figure 5: Lancement de NetBeans

### 4. Configurer le JDK :

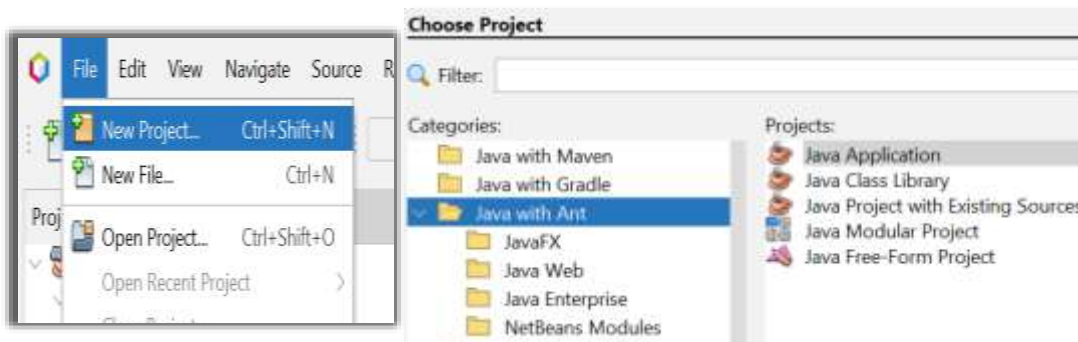
- Lors du premier lancement, NetBeans peut automatiquement détecter le JDK. Sinon, vous pouvez le configurer dans Tools > Java Platforms.



## 5. Créer un nouveau projet :

Nous voici en train d'utiliser l'environnement de développement intégré NetBeans, à savoir qu'un fichier java est un Projet.

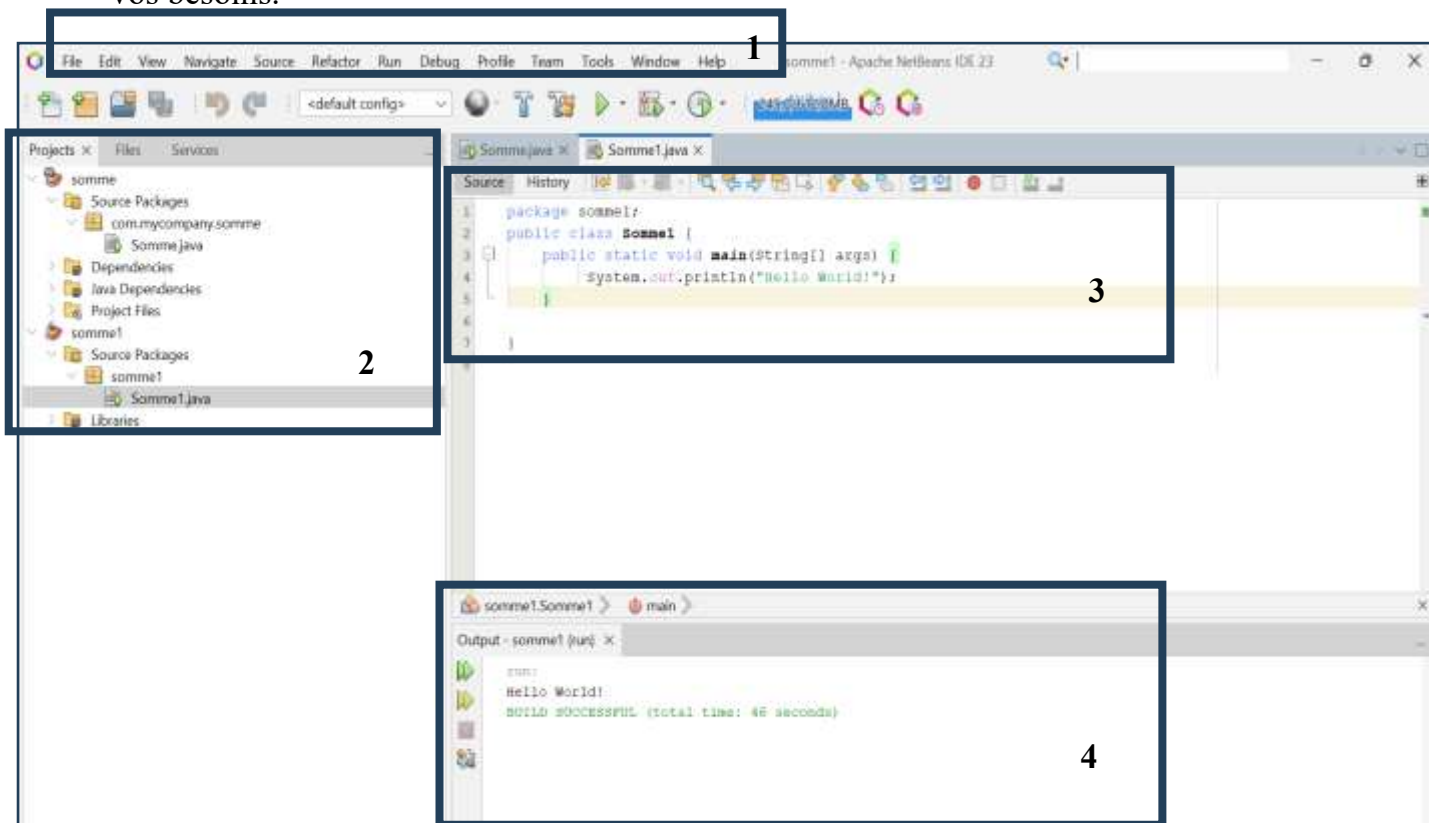
- Sélectionnez File > New Project, choisissez « Java » et suivez les instructions.



## 6. Personnaliser l'IDE :

Les options peuvent légèrement varier selon la version de NetBeans que vous utilisez, mais les étapes générales restent les mêmes. N'hésitez pas à explorer les

différentes sections pour personnaliser votre environnement de développement selon vos besoins.



## 2. BASES DE JAVA

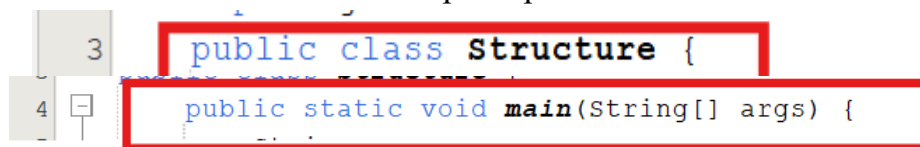
### 2.1. SYNTAXE DE BASE

#### 2.1.1. Structure d'un programme Java

Un **programme** est une série d'instructions écrites dans un fichier. En Java, une instruction se termine par un point-virgule (;).

Un programme Java est relativement simple et suit un schéma standard. Voici les principaux éléments d'un programme Java :

- **Package** (facultatif) : Définit le package dans lequel la classe est contenue.
- **Importations** (facultatif) : Permet d'importer d'autres classes ou bibliothèques.
- **Classe** : La définition de la classe principale.



```

3 public class Structure {
4 public static void main(String[] args) {

```

- **Méthode principale** : Le point d'entrée du programme.

Un programme java est composé d'au moins une classe, cette classe doit contenir une méthode main, qui est la méthode principale du programme, c'est elle que la jvm va chercher afin de lancer votre programme.

Une méthode est une suite d'instruction à exécuter.

Une méthode contient :

Un entête : identité de la méthode

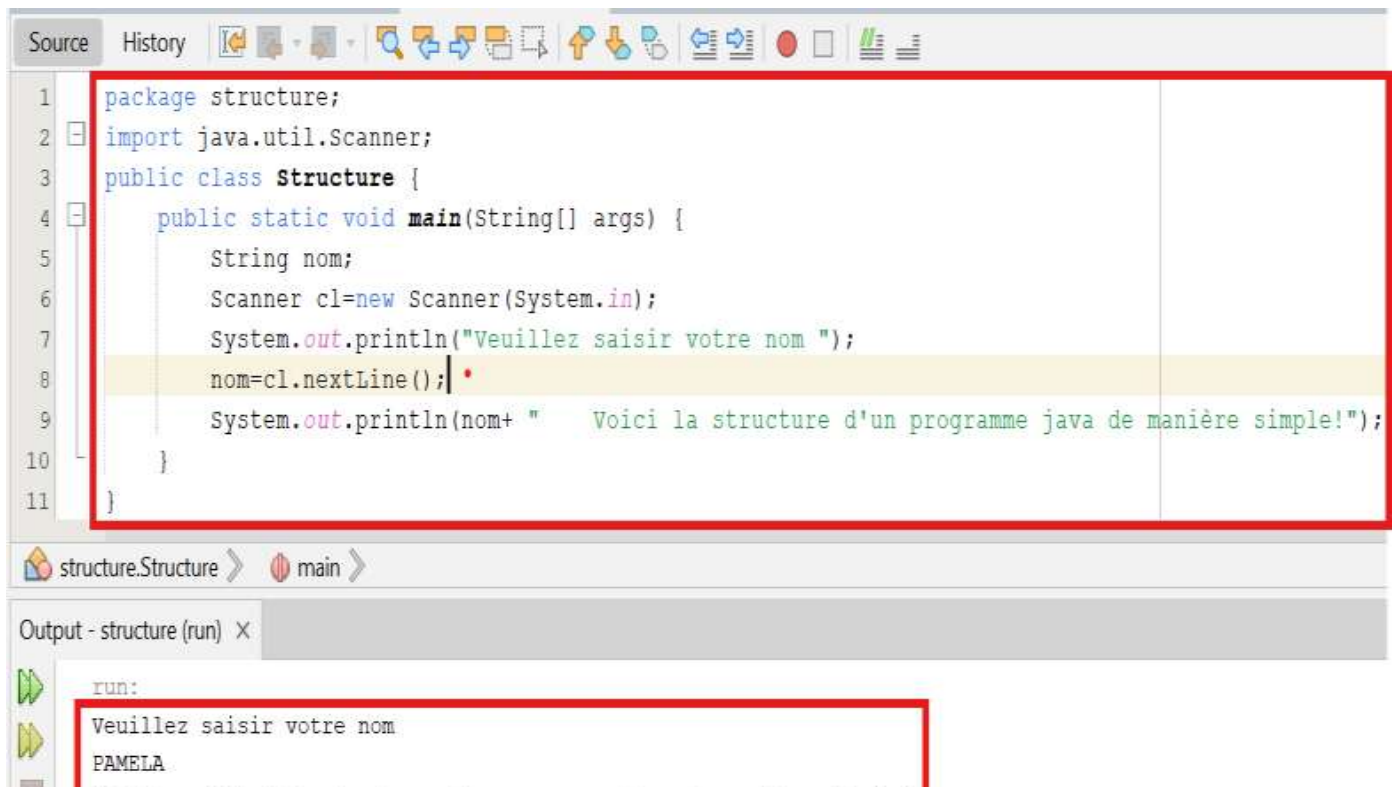
Un corps : entre accolade {...}

Une valeur de retour : seule la méthode void qui ne renvoie rien.

A partir de maintenant jusqu'à ce que nous parlions de la programmation orienté objet, nous allons faire ce qu'on appelle des **programmes impératifs ou procéduraux**.

Voici un programme complet d'un programme procédural en java

### 2.1.2. Variables et constantes



The screenshot shows an IDE with a source editor and an output console. The source editor contains the following Java code:

```

1 package structure;
2 import java.util.Scanner;
3 public class Structure {
4     public static void main(String[] args) {
5         String nom;
6         Scanner cl=new Scanner(System.in);
7         System.out.println("Veuillez saisir votre nom ");
8         nom=cl.nextLine();
9         System.out.println(nom+ "    Voici la structure d'un programme java de manière simple!");
10    }
11 }

```

The output console shows the execution of the program:

```

run:
Veuillez saisir votre nom
PAMELA

```

Figure 6: Ecriture et compilation d'un programme java

Une variable représente une donnée modifiable. Elle est logée dans un emplacement mémoire repéré par une adresse. Une Variable est une case mémoire réservée pour stocker une information.

Cependant, une constante est une donnée NON modifiée. Elle est aussi logée dans un emplacement mémoire.

En java, nous avons deux types de variables :

- Types simples ou « primitif » ;
- Types complexes ou encore des objets.

Ce qu'on appelle des types simples, en java ce sont tout bonnement des nombres, des réels, des booléens ou encore des caractères. Mais vous allez voir qu'il a plusieurs façons de déclarer certain de ces types.

| Type de données | Taille en mémoire           | Plage des valeurs                                                                                                                                                                                                        | Description                                   |
|-----------------|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Boolean         | 2 oct                       | True – False                                                                                                                                                                                                             | Vrai et Faux                                  |
| Byte            | 1 oct                       | 0 à 255                                                                                                                                                                                                                  | Suite de bits                                 |
| String          | 10 oct + longueur de chaîne | 0 à environ 2 milliards                                                                                                                                                                                                  | Chaînes de caractères                         |
| Short           | 2oct                        | De -32.768 à 32.767                                                                                                                                                                                                      |                                               |
| Int             | 4 oct                       | de -2.147.483.648 à 2.147.483.647                                                                                                                                                                                        | Entiers                                       |
| Long            | 8 oct                       | De -2.147.483.648 à 2.147.483.647                                                                                                                                                                                        | Entiers longs                                 |
| Float           | 4 oct                       | De -3,402823.10 <sup>38</sup> à -1,401298.10 <sup>-45</sup><br>pour les valeurs négatives<br>De 1,401298.10 <sup>-45</sup> à 3,402823.10 <sup>38</sup><br>pour les valeurs positives                                     | Nombres à virgule flottante, simple précision |
| Double          | 8 oct                       | De -1,79769313486232.10 <sup>308</sup> à -4,94065645841247.10 <sup>-324</sup><br>pour les valeurs négatives<br>De 4,94065645841247.10 <sup>-324</sup> à 1,79769313486232.10 <sup>308</sup><br>pour les valeurs positives | Nombres à virgule flottante, double précision |
| Char            | 2 oct                       | un caractère Unicode                                                                                                                                                                                                     | un caractère Unicode                          |

Les règles générales pour nommer les variables sont :

- Les noms peuvent contenir des lettres, des chiffres, des sous-trait ( et des signes de dollar
- Les noms doivent commencer par une lettre
- Les noms doivent commencer par une lettre minuscule et ne peuvent pas contenir d'espaces blancs
- Les noms peuvent aussi commencer par \$ et \_
- Les noms sont sensibles à la casse (« myVar » et « myvar » sont des variables différentes)
- Les mots réservés (comme les mots-clés Java, tels que int, boolean...) ne peuvent pas être utilisés comme noms.
- Tout ceci sans accent !!!

Note : Tous vos noms de classes doivent commencer par une majuscule et une constante commence par le mot clé final

Ex : public class Mbuyi {} ;

```
String jour ;
int nombre ;
double nombreEntier ;
```

## 2.2. OPERATEURS

### 2.2.1. Opérateurs arithmétiques

| Opérateurs | Noms           | opérateurs |                           |
|------------|----------------|------------|---------------------------|
| +          | Addition       | +=         | Assignment additive       |
| -          | Soustraction   | -=         | Assignment soustractive   |
| *          | Multiplication | *=         | Assignment multiplicative |
| /          | Division       | /=         | Assignment divisionnelle  |
| %          | Modulo         | %=         | Assignment modulo         |
| ++         | Incrémentation | --         | Décrémentation            |

### 2.2.2. Opérateurs de la concaténation

L'opérateur de concaténation en Java est le symbole « + » utilisé pour concaténer des chaînes de caractères.

```

1  package opérateurs;
2  public class Opérateurs {
3      public static void main(String[] args) {
4          String nom1 = "John";
5          String nom2 = "Misangi";
6
7          String nom = nom1 + " " + nom2;
8          System.out.println("Nom complet: " + nom);
9      }
10
11  }
```

Output - opérateurs (run) ×

```

run:
Nom complet: John Misangi
BUILD SUCCESSFUL (total time: 0 seconds)
```

### 2.2.3. Opérateurs d'affectation

Ah, l'opérateur d'affectation en Java est le symbole « = » qui est utilisé pour assigner une valeur à une variable. L'interface ci-haut illustre son utilisation.

### 2.2.4. Opérateurs de comparaison ou relationnel

Les opérateurs relationnels en Java sont utilisés pour comparer des valeurs et retourner un résultat booléen (vrai ou faux). Voici une liste des opérateurs relationnels les plus couramment utilisés en Java :

| Opérateurs | Résultats            |
|------------|----------------------|
| ==         | Egale                |
| !=         | Différent            |
| <          | Inférieur            |
| <=         | Inférieur ou égale à |
| >          | Supérieur            |
| >=         | Supérieur ou égale à |

- Seuls les types numériques peuvent être comparés avec les opérateurs. D'ordre entiers, flottants et caractères peuvent être comparés avec les opérateurs d'ordre strict.
- Chaque opérateur renvoie un type booléen.

### 2.2.5. Opérateurs logiques ou Booléens

En Java, les opérateurs logiques servent à réaliser des tâches logiques sur des expressions logiques.

| Opérateurs | Résultats                    |
|------------|------------------------------|
| &          | Et logique                   |
| &=         | Assignment avec ET           |
| !          | Ou logique                   |
| !=         | Assignment avec Ou           |
| ^          | Ou exclusive                 |
| ^=         | Assignment avec OU exclusive |
|            | OU avec court-circuit        |
| &&         | ET avec court-circuit        |

## 2.3. LES CONVERSIONS OU LE CAST

La hiérarchie de conversion permettant de ne pas dénaturer la valeur initiale :

int → long → float → double.

Note : par défaut, pour les types byte, char, short, si un de ces types apparait dans une expression, la valeur est d'abord convertie en int.

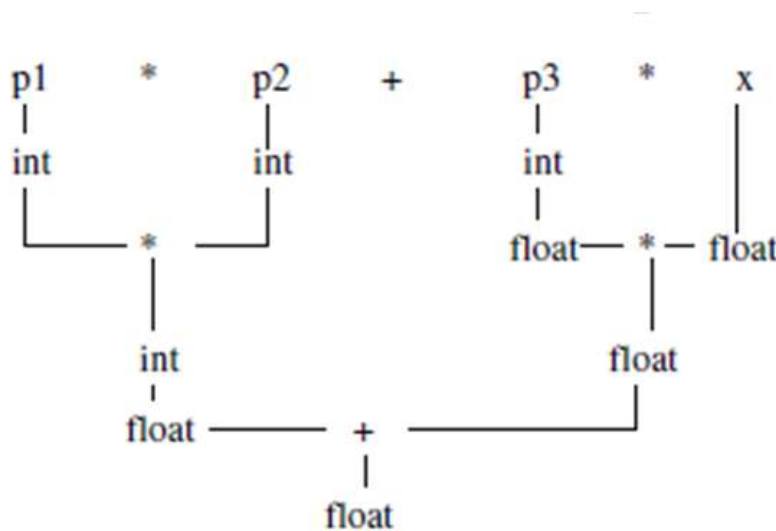
Exemple :

short p1=1, p2=1, p3=1;

float x=2;

L'évaluation de  $p1 * p2 + p3 * x$  est faite comme le montre suite :

Ce type de conversion s'appelle une conversion d'ajustement ou Cast de variable.



### 2.3. 1. Conversion d'un type vers un autre type numérique

Vous l'avez vu : nous pouvons passer directement d'un int à un type float. Mais ceci ne fonctionnera pas dans le sens contraire (inverse) sans une perte de précision. Lorsque nous « castons » un double en int ou de float en int ; la valeur de ce double ou de float est troquée. Ce qui signifie que l'int en question prendra la valeur entière du double quelle que soit la valeur des décimales.

double i=1.923 ;

int k=i ; ici nous aurons comme résultat 1.

Sachez que l'affectation, le calcul, le cast, le test, l'incrément... toutes ces choses sont des opérations ! et java les fait dans un certain ordre, suivant une priorité.

- Un calcul ;

- Un cast de valeur de résultat de calcul ;
- Une affectation dans la variable résultat ;

### 2.3.2. Conversion d'un type numérique en String vise versa

Pour convertir un nombre en chaîne de caractères en Java, vous pouvez utiliser la méthode `String.valueOf()` ou concaténer le nombre avec une chaîne vide.

Voici comment vous pouvez convertir un nombre en chaîne de caractères en Java :

#### 1. Utilisation de la méthode `String.valueOf()` :

```
int a= 123;
```

```
String nom = String.valueOf(a);
```

Il est aussi possible de créer un objet de la classe `String`

```
int i=12 ;
```

```
String p= new String ();
```

```
p=p.valueOf(i) ;
```

`p` est donc une variable de type `String` contenant de caractère 12. Ceci fonctionne aussi avec les autres types numériques. La méthode `valueOf()` de la classe `String` renvoie une représentation chaîne pour les types primitifs.

#### 2. Concaténation avec une chaîne vide

```
double x = 3.674;
```

```
String y= « » + x;
```

### 2.3.3. Conversion une chaîne de caractères en un type numérique

Il est possible de convertir une chaîne de caractère en numérique en java, pour convertir une chaîne en numérique il est important d'utiliser les méthodes :

#### 1. Conversion de chaîne en entier (int) :

```
String strEntier = « 123 »;
```

```
int entier = Integer.parseInt(strEntier);
```

#### 2. Conversion de chaîne en double

```
String st = "3.14"; double z= Double.parseDouble(st);
```

Assurez-vous que la chaîne que vous essayez de convertir en numérique a un format valide pour le type de données auquel vous essayez de la convertir. Sinon, une exception pourrait être levée lors de la conversion.

## 2.3. LES ENTREES CLAVIER

Un programme devient encore plus intéressant si l'utilisateur peut lui-même saisir les données. Ici, nous allons utiliser un objet Scanner.

### 2.3.1. Utilisation la classe Scanner (Objet Scanner)

Scanner en Java est une classe très utile qui permet de lire des entrées à partir de différentes sources de données, telles que la console, des fichiers ou des chaînes de caractères. Voici quelques points importants à retenir sur l'utilisation de l'objet Scanner :

#### Création de l'Objet Scanner

Pour commencer à lire les entrées de l'utilisateur, vous devez créer un objet Scanner en spécifiant la source de données à partir de laquelle vous souhaitez lire (par exemple System.in pour la console).

```
package structure;  
import java.util.Scanner;  
public class Structure {  
    public static void main(String[] args) {  
        String nom;  
        Scanner cl=new Scanner(System.in);
```

#### Méthodes de lecture

L'objet Scanner offre diverses méthodes pour lire différents types de données, tels que nextInt() pour les entiers, nextDouble() pour les doubles, nextLine() pour les chaînes de caractères, etc. Assurez-vous d'utiliser la méthode appropriée en fonction du type de données que vous attendez.

```
Scanner scEntier = new Scanner (System.in);  
Scanner scChaine = new Scanner (System.in);
```

```

System.out.print(« Entrez votre âge : « );
int age = scEntier.nextInt();
System.out.print(« Entrez votre nom : « );
String nom = scChaine.nextLine();
System.out.println( nom+ « Vous avez « +age+ » ans « );

```

### Fermeture de l'objet Scanner

Une fois que vous avez terminé de lire les entrées de l'utilisateur, il est recommandé de fermer l'objet Scanner en appelant la méthode `close()`. Cela libère les ressources associées à l'objet Scanner et garantit une bonne gestion des ressources.

```

scEntier.close();
scChaine.close();

```

### 2.3.2. BufferedReader

`BufferedReader` est une classe qui nous aide à lire les entrées utilisateur depuis la console de manière efficace et robuste. Il est souvent utilisé pour lire des données ligne par ligne à partir de flux d'entrée, comme `System.in`.

```

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;
BufferedReader baba = new BufferedReader(new InputStreamReader(System.in));

```

Pour lire une ligne de texte avec Baba `BufferedReader`, vous pouvez utiliser la méthode `readLine()` comme ceci :

```
String line = baba.readLine();
```

Avec `BufferedReader`, vous lisez les données sous forme de chaînes de caractères à l'aide de la méthode `readLine()`, qui lit une ligne entière à la fois. Ensuite, vous pouvez convertir ces chaînes en différents types de données en utilisant des méthodes de conversion appropriées, telles que `Integer.parseInt()` pour les entiers ou `Double.parseDouble()` pour les décimaux.

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

BufferedReader baba = new BufferedReader(new InputStreamReader(System.in));
try {
    String input = baba.readLine();
    int entier = Integer.parseInt(input);
    System.out.println("Entier lu: " + entier);
} catch (IOException e) {
    System.out.println("Une erreur s'est produite lors de la lecture.");
}
```

**java.io.BufferedReader** : La classe `BufferedReader` est utilisée pour lire des données texte à partir d'un flux d'entrée caractère. Elle offre des méthodes pour lire des lignes de texte et des caractères, ce qui la rend très utile pour la lecture de données depuis des sources telles que `System.in` (entrée standard) ou des fichiers.

**java.io.InputStreamReader** : La classe `InputStreamReader` sert à convertir les octets lus à partir d'un flux d'entrée en caractères. Elle prend un flux d'entrée binaire (comme `System.in`) et le convertit en un flux de caractères, ce qui permet à des classes comme `BufferedReader` de lire les données sous forme de texte.

**java.io.IOException** : `IOException` est une exception qui est généralement levée lorsqu'il y a un problème lors de l'entrée/sortie de données, comme une erreur de lecture ou d'écriture. C'est une bonne pratique de gérer les exceptions `IOException` lors de la lecture de données, afin de gérer les erreurs de manière appropriée.

## 3. Structure de Contrôle

### 3.1. CONDITION

Les structures conditionnelles sont essentielles pour contrôler le flux d'exécution de votre programme en fonction de différentes conditions. Assurez-vous de comprendre comment et quand utiliser chaque type de structure conditionnelle pour écrire un code clair et logique.

#### 3.1.1. Structure conditionnelle if

##### IF

Permet d'exécuter un bloc de code si une condition est vraie.

Syntaxe

```
if (condition) {  
    Bloc de code à exécuter si la condition est vraie }
```

Exemple : Ecrire un programme java permettant d'afficher le message x est supérieur à 5 si la valeur de X est supérieure à 5.

code

```
int x = 10;
```

```
if (x > 5) {  
    System.out.println(« x est supérieur à 5 »); }
```

##### If-else

L'instruction else est utilisée avec if pour exécuter un bloc de code alternatif si la condition if est fausse.

```
if (condition) {  
    Bloc de code à exécuter si la condition est vraie  
} else {Bloc de code à exécuter si la condition est fausse  
}
```

### Exemple

```
int y = JOptionPane.showInputDialog(null, "Saisir un nombre")
if (y > 5) {
    System.out.println("y est supérieur à 5 »); }
Else {
    System.out.println("y est inférieur ou égal à 5 »); }
```

### if-else if-else

Permet de gérer plusieurs conditions de manière séquentielle et m'exécuter qu'un

if (condition1) {

Bloc de code à exécuter si condition1 est vraie }

else if (condition2) {

Bloc de code à exécuter si condition2 est vraie }

else {

Bloc de code à exécuter si aucune des conditions précédentes n'est vraie }

### Exemple

```
int y = Integer.parseInt(JOptionPane.showInputDialog(null, "Saisir un nombre"))
if (y > 5) {
    System.out.println(y+ » est supérieur à 5 »);
}
else if(y<5){
    System.out.println(y+ » est inférieur à 5 »);
}
else {
    System.out.println("y est inférieur ou égal à 5 »);}
```

Ici on peut aller à plusieurs conditions avec else if et lorsqu'on arrive un très grand nombre de condition on utilisera l'instruction switch

### 3.1.2. La structure conditionnelle switch

La structure switch est utilisée pour évaluer une expression et exécuter différents blocs de code en fonction de la valeur de cette expression. Elle est souvent utilisée comme une alternative aux longues séries d'instructions if-else.

La structure switch est utilisée pour effectuer différentes actions en fonction de la valeur d'une variable. Voici quelques points importants à retenir sur la structure switch en Java :

#### 1. Syntaxe de la structure switch :

- La structure switch est composée d'une expression (variable) et de plusieurs cas (case) pour comparer la valeur de cette expression.

#### 2. Cas (case) et Déclaration default :

- Chaque cas case dans un bloc switch correspond à une valeur possible de l'expression.
- La déclaration default est exécutée si aucun des cas case ne correspond à la valeur de l'expression.

#### 3. Break Statement :

- Après l'exécution d'un cas case, il est important d'inclure l'instruction break pour sortir du bloc switch.
- Sinon, le flux d'exécution continuera à l'instruction suivante, ce qui peut entraîner un comportement inattendu.

```
switch (expression) {
case valeur1:
    Code à exécuter si expression == valeur1
    break;
case valeur2:
    Code à exécuter si expression == valeur2
```

*break;*

*Autres cas possibles*

*default:*

*Code à exécuter si aucune des valeurs ne correspond à l'expression }*

## 3.2. BOUCLES ('FOR', 'WHILE', 'DO-WHILE')

Le rôle des boucles est de répéter un certain nombre de fois les mêmes opérations.

Tous les programmes, ou presque, ont besoin de ce type de fonctionnalité.

Nous utiliserons les boucles pour permettre à un programme de recommencer depuis le début, pour attendre une action précise de l'utilisateur, parcourir une série de données, etc.

Une boucle s'exécute tant qu'une condition est remplie. Nous réutiliserons donc des notions des conditions

### 3.2.1. La boucle while



Une boucle commence par une déclaration : ici `while`. Cela veut dire, à peu de chose près, « tant que ». Puis nous avons une condition : c'est elle qui permet à la boucle de s'arrêter. Une boucle n'est utile que lorsque nous pouvons la contrôler, et donc lui faire répéter une instruction un certain nombre de fois. C'est à ça que servent les conditions. Ensuite nous avons une ou plusieurs instructions : c'est ce que va répéter notre boucle !

Syntaxe :

```
while (Condition ){
Instructions à répéter
}
```

Exemple : écrire un programme java qui demande à l'utilisateur de saisir son prénom et qu'en suite le programme affiche « Bonjour <PRENOM> » et enfin que le programme lui demande s'il veut quitter ou pas le programme.

## Résolution

```
package opérateurs;
import java.util.Scanner;
public class BouclWhile {
    public static void main(String[] args) {
        String prenom;
        String reponse = "O";
        Scanner sc = new Scanner(System.in);
        while ((reponse.equalsIgnoreCase("O")) || (reponse.equalsIgnoreCase("Oui"))){
            System.out.println("Donnez un prénom : ");
            prenom = sc.nextLine();
            System.out.println("Bonjour " + prenom + ", comment vas-tu ?");
            System.out.println("Voulez-vous réessayer ? (O/N)");
            reponse = sc.nextLine();
        }
    }
}
```

### On peut même mettre une boucle dans une autre boucle

Ecrire un programme java qui demande à l'utilisateur de saisir son prénom et son nom ensuite que le programme affiche « Bonjour <PRENOM> <NOM> » et enfin que le programme lui demande s'il veut quitter ou pas le programme tant que l'utilisateur saisira autre chose que oui ou non que le programme redemande de saisir oui(O) ou non(N).

```

package opérateurs;
import java.util.Scanner;
public class While2 {
    public static void main(String[] args) {
        String prenom;
        String reponse = "O";
        Scanner sc = new Scanner(System.in);
        while ((reponse.equalsIgnoreCase("O")) || (reponse.equalsIgnoreCase("Oui")))
        {
            System.out.println("Donnez un prénom : ");
            prenom = sc.nextLine();
            System.out.println("Bonjour " + prenom + ", comment vas-tu ?");
            reponse = "";

            while(! (reponse.equalsIgnoreCase("O")) && (!reponse.equalsIgnoreCase("N")))
            {
                System.out.println("Voulez-vous réessayer ? (O/N)");
                reponse = sc.nextLine();
            }
            System.out.println("Au revoir...");
        }
    }
}

```

Ecrire un programme java qui demande un nombre compris entre 10 et 20, jusqu'à ce que la réponse convienne. En cas de réponse supérieure à 20, on fera apparaître un

10.

```

package opérateurs;
import java.util.Scanner;
public class While2 {
    public static void main(String[] args) {
        String prenom;
        String reponse = "O";
        Scanner sc = new Scanner(System.in);
        while ((reponse.equalsIgnoreCase("O")) || (reponse.equalsIgnoreCase("Oui")))
        {
            System.out.println("Donnez un prénom : ");
            prenom = sc.nextLine();
            System.out.println("Bonjour " + prenom + ", comment vas-tu ?");
            reponse = "";

            while(! (reponse.equalsIgnoreCase("O")) && (!reponse.equalsIgnoreCase("N")))
            {
                System.out.println("Voulez-vous réessayer ? (O/N)");
                reponse = sc.nextLine();
            }
            System.out.println("Au revoir...");
        }
    }
}

```



### 3.2.2. do...while

La boucle do...while fonctionne quasiment de la même manière que la boucle while à deux petites différences près :

- La boucle **do... while** s'exécutera *au moins une fois* ;
- Le point virgule qui se situe après la condition du **while**.

```

    public static void main(String[] args) {
        String prenom = new String();
        char reponse = ' ';
        Scanner sc = new Scanner(System.in);
        do{
            System.out.println("Donnez un prénom : ");
            prenom = sc.nextLine();
            System.out.println("Bonjour " + prenom + ", comment vas-tu ?");
            do{
                System.out.println("Voulez-vous réessayer ? (O/N)");
                reponse = sc.nextLine().charAt(0);
            }while(reponse != 'O' && reponse != 'N');
        }while (reponse == 'O');
        System.out.println("Au revoir...");
    }
}

```

### 3.2.3. for

Cette boucle est un peu particulière puisqu'elle prend trois attributs dans sa condition et agit en conséquence. Je m'explique : jusqu'ici, nous avons fait des boucles avec :

- Déclaration d'une variable avant la boucle ;
- Initialisation de cette variable ;
- Mise à jour (Incrémentatation ou décrémentation) de celle-ci dans la boucle.

Eh bien on met tout ça dans la condition de la boucle for, Tous ceci sont séparés par un point-virgule ( 😊 ; 😊 ; 😊 ) et c'est tout.

Syntaxe

*for (initialisation; condition; mise à jour) {*

*Bloc de code à répéter*

*}*

Exemple : Ecrire un programme java permettant à l'utilisateur de d'afficher N fois « Voici la nième ligne ».

```
package opérateurs;
public class For1 {
    public static void main(String[] args) {
        for(int i = 1; i <= 10; i++)
            System.out.println("Voici la ligne "+i);
    }
}
```

```
1
Voici la ligne 1
Voici la ligne 2
Voici la ligne 3
Voici la ligne 4
Voici la ligne 5
Voici la ligne 6
Voici la ligne 7
Voici la ligne 8
Voici la ligne 9
Voici la ligne 10
```

Avec la boucle for le nombre d'itération est connu au préalable.

### 3.3. EXERCICES

## 4. Tableaux et Collections

Un tableau est une variable particulière qui stock au moins deux (2) valeurs. Les tableaux en Java sont de taille fixe une fois qu'ils sont créés, mais vous pouvez modifier les valeurs de leurs éléments individuellement.

Un tableau est une structure de données qui permet de stocker une collection d'éléments de même type. Chaque élément d'un tableau est accessible à l'aide d'un index ou indice. Assurez-vous de faire attention aux indices pour éviter les dépassements de tableau.

### 4.1. TABLEAU A UNE DIMENSION

|        |   |    |    |    |    |   |   |    |     |
|--------|---|----|----|----|----|---|---|----|-----|
| Valeur | 5 | 12 | 13 | 23 | 34 | 4 | 5 | 13 | 239 |
|--------|---|----|----|----|----|---|---|----|-----|

#### Déclaration et utilisation des tableaux

Une variable de type int ne peut pas recevoir une chaîne de caractères. Il en va de même pour les tableaux. Voyons tout de suite comment ils se déclarent :

*<type du tableau> <nom du tableau> [] = { <contenu du tableau> };*

La déclaration ressemble beaucoup à celle d'une variable quelconque, si ce n'est la présence de crochets [] après le nom de notre tableau et d'accolades {} encadrant l'initialisation de celui-ci.

Dans la pratique, ça nous donnerait quelque chose comme ceci :

```
char tabC[] = {'p','a','m','e','l','a'};
```

```
int z[] = {5,12,13,23,34,4,5,13,239};
```

l'indice est l'ordre d'élément dans un tableau et cet ordre commence par zéro (0)

|        |   |    |    |    |    |   |   |    |     |
|--------|---|----|----|----|----|---|---|----|-----|
| Indice | 0 | 1  | 2  | 3  | 4  | 5 | 6 | 7  | 8   |
| Valeur | 5 | 12 | 13 | 23 | 34 | 4 | 5 | 13 | 239 |

```
double tableauDouble[] = {5.0,12.0,13.45,23.0,34.89,4.0,5.9,13.7,239.4};
```

```
String tableauChaine [] = { »girmela », « girla », « pamiresse », « meliresse »};
```

Vous pouvez aussi déclarer un tableau vide, mais celui-ci devra impérativement contenir un nombre de cases bien défini.

```
int vente[] = new int[6];
```

Ou encore

```
int[] tableauEntier2 = new int[6];
```

Cette opération est très simple, car vraiment ressemblante à ce que vous faisiez avec vos variables ; je vous propose donc tout de suite de nous pencher sur une belle variante de ces tableaux. . .

La méthode `length` vous permet de connaître la taille du tableau.

## Boucle des tableaux

### FOR

Vous pouvez boucler les éléments du tableau avec la boucle, et utiliser la propriété pour spécifier combien de fois la boucle doit s'exécuter. `for length`

Cet exemple crée un tableau de chaînes de caractères puis utilise une boucle pour imprimer chaque élément, un par un : `for`

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};

for (int i = 0; i < cars.length; i++) {
    System.out.println(cars[i]);
}
```

### FOR EACH

Il existe également une boucle « **for-each** », utilisée exclusivement pour faire passer en boucle des éléments d'un tableau (ou d'autres structures de données) , Vous pouvez donc lire la boucle comme : « *pour chaque variable dans le tableau* » et la boucle est important pour imprimer les éléments du tableau.

Crée une variable qui a le même type que son tableau, après deux points rajouter le nom du tableau.

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};

for (String car : cars) {
    System.out.println(car);
}
```

Enfin imprimer la variable déclarée dans la boucle for.

**Note :** La boucle **for-each** est très utile quand on n'a besoin que de lire des éléments. Si vous souhaitez modifier les éléments plus tard, ou suivre leur index, utilisez plutôt une boucle régulière.for.

## 4.2. TABLEAU A DEUX DIMENSION

Un tableau multidimensionnel contient au moins deux tableaux c'est-dire des lignes et les colonnes

Indice Ici l'indice porte le numéro de la ligne suivi de la colonne(lico) 0.0, 1.0...

|           |   |    |    |    |    |     |     |    |     |
|-----------|---|----|----|----|----|-----|-----|----|-----|
| Vente     | 5 | 12 | 13 | 23 | 34 | 4   | 5   | 13 | 239 |
| Publicité | 4 | 56 | 13 | 19 | 0  | 112 | 562 | 98 | 1   |

Un tableau à deux dimensions se déclare avec double crochets [][] le premier crochet représente le nombre de ligne du tableau et le deuxième représente le nombre de colonne et pour déclarer l'illustration ci-haut :

Ici on créer et on l'initialise

```
int vente[][]={ {5,12,13,23,34,4,5,13,239} {4,56,13,19,0,112,562,98,1} }
```

Ici on créer le tableau sans pourtant initialiser.

```
int vente [][]=new int[2][9] ;
```

### Exemple

```
public static void main(String[] args) {
    Scanner sc=new Scanner(System.in);
    Scanner sd=new Scanner(System.in);
    System.out.println ("saisir le nombre d'article");
    int art=sc.nextInt();
    String tab [][]= new String [art][4];
    for(int i=0;i<art;i++){
        System.out.print("Saisir la designation   ");
        tab[i][0]=sd.nextLine();
        System.out.print("Saisr la Qte       ");
        tab[i][1]=sd.nextLine();
        System.out.print("Saisr la pu       ");
        tab[i][2]=sd.nextLine();
        tab[i][3] =String.valueOf(Double.parseDouble (tab[i][1])*Double.parseDouble (tab[i][2]));
        System.out.print("\n");
    }
    for(int i=0;i<art;i++){
        System.out.print("\t"+tab[i][0]+"\t");
        System.out.print(tab[i][1]+"\t");
        System.out.print(tab[i][2]+"\t");
        System.out.print(tab[i][3]+"\n");
    }
}
```

saisir le nombre d'article

2

Saisir la designation      SAVON

Saisr la Qte      3

Saisr la pu      56

Saisir la designation      GERMOL

Saisr la Qte      12

Saisr la pu      45

|       |   |    |       |
|-------|---|----|-------|
| SAVON | 3 | 56 | 168.0 |
|-------|---|----|-------|

|        |    |    |       |
|--------|----|----|-------|
| GERMOL | 12 | 45 | 540.0 |
|--------|----|----|-------|

---

les marchandises vendues ne sont ni reprises, ni changées p.  
BUILD SUCCESSFUL (total time: 1 minute 16 seconds)

- 1) Ecrire un programme java qui déclare et remplisse un tableau de 7 valeurs numériques en les mettant toutes à zéro.
- 2) Ecrire un programme java qui déclare et remplisse un tableau contenant les six voyelles de l'alphabet français.

- 3) Ecrire un programme qui déclare un tableau de 9 notes, dont on fait ensuite saisir les valeurs par l'utilisateur et enfin calculer la moyenne de ces notes.

## 4.3. INTRODUCTION AUX COLLECTIONS

Les structures de données sont des moyens de stocker et d'organiser les données pour pouvoir les utiliser efficacement.

Un tableau est un exemple de structure de données, qui permet de stocker plusieurs éléments dans une seule variable.

Java inclut également de nombreuses autres structures de données dans le package. Chacun est utilisé pour traiter les données de différentes manières. `java.util`

Voici quelques-unes des plus courantes :

- `ArrayList`
- `HashSet`
- `HashMap`

Il est important de retenir qu'un tableau en java est fixe c'est-à-dire qu'on peut bien changer ses valeurs mais pas sa taille raison pour laquelle on va voir nos deux classes **`ArrayList`** et **`HashMap`** qui vont nous aider à avoir des listes dynamiques.

L'**`ArrayList`** est une classe de la bibliothèque `java.util` qui implémente l'interface<sup>2</sup> `List`. Cela signifie qu'elle permet de stocker une collection dynamique d'éléments de manière ordonnée. Elle stocke des éléments de n'importe quel type (Objet, primitives, etc...), sa taille peut varier (dynamiquement) et permet de stocker les éléments en doublant.

---

<sup>2</sup> Interface : cette notion sera développée au chapitre de la programmation orientée objet.

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> cars = new ArrayList<String>();
        cars.add("Volvo");
        cars.add("BMW");
        cars.add("Ford");
        cars.add("Mazda");
        System.out.println(cars);
    }
}
```

## HashSet

Est une collection où chaque élément est unique – aucun doublon n'est autorisé.

## HashMap

Est une classe de la bibliothèque standard Java qui implémente l'interface Map. Il permet de stocker des paires clé-valeur, où chaque clé est unique et associée à une seule valeur.

```
import java.util.HashMap;

public class Pays {
    public static void main(String[] args) {

        HashMap<String, String> capitalpays = new HashMap<String, String>();

        capitalpays.put("England", "London");
        capitalpays.put("Germany", "Berlin");
        capitalpays.put("Norway", "Oslo");
        capitalpays.put("USA", "Washington DC");
        System.out.println(capitalCities);
    }
}
```

## 5. METHODES DE CLASSE

Ce chapitre aura pour but de vous faire découvrir la notion de méthode (on l'appelle fonction dans d'autres langages comme dans les tableurs). Vous en avez peut-être déjà utilisé.

Le principal avantage des méthodes est de pouvoir factoriser le code : grâce à elles, vous n'avez qu'un seul endroit où effectuer des modifications lorsqu'elles sont nécessaires. J'espère que vous comprenez mieux son intérêt. Vous vous rendez sûrement compte qu'il y a certaines choses que vous effectuez souvent. Plutôt que de recopier sans arrêt les mêmes morceaux de code, vous pouvez écrire une méthode. . .

Il existe plusieurs méthodes dans le langage JAVA, présentes dans la classe String de la bibliothèque java.lang que nous allons utiliser et bien d'autres on peut catégoriser les méthodes en deux :

- Les natives (prédéfinies) ;
- Personnalisées (les nôtres).

### 5.1. LES METHODES NATIVES

Ici, il est quasiment impossible d'apprendre toutes les méthodes natives de java dans un seul chapitre.

#### **toLowerCase()**

Cette méthode permet de transformer toute saisie clavier de type caractère en minuscules. Elle n'a aucun effet sur les nombres et il a comme rival **toUpperCase()**.

Ex : String min=«COUCOU TOUT EST GRACE »

String maj=min.toLowerCase () ;

#### **Concat ()**

Permet de concaténer deux chaînes de caractères

Pam=emmanuel.concat(jon

iko) ;

**length()**

Permet de donner la longueur d'une chaîne de caractère (en comptant les espaces blancs).

**Equals()**

Permet de voir si deux chaînes de caractères sont identiques en tenant compte de la case. Donc de faire des tests

Ex : `if (str1.equals (str2))`

On peut même demander la non-vérification de l'égalité grâce à l'opération de négation (!)

Ex: `if (!str1.equals (str2))`

Cette méthode renvoi un booléen. C'est pourquoi nous pouvons utiliser cette fonction dans les tests de condition. Il a son frère **equalsIgnoreCase(chaine)** compare deux chaînes sans tenir compte de la case.

**charAt ()**

Le résultat de cette méthode sera un caractère, car il s'agit d'une méthode d'extraction de caractère, elle ne s'opère que pour le String ! Elle possède la même particularité que les tableaux, c'est-à-dire que, pour cette méthode le premier caractère sera le numéro 0. Cette méthode prend un entier comme argument.

`String car= « générer » ;`

`char ran=car.charAt(5) ;`

**substring()**

Comme son nom l'indique, elle permet d'extraire une sous-chaîne de caractère. Cette méthode prend 2 entiers comme arguments. Le premier définit le début de la sous chaîne à extraire inclus. Le deuxième correspond au dernier caractère à extraire exclus. Et le premier caractère est aussi le num 0.

Les méthodes listées ci-dessous nécessitent la classe Math, présente dans java.lang. Elle fait donc partie des fondements du langage. Par conséquent, aucun import particulier n'est nécessaire pour utiliser la classe Math qui regorge de méthodes utiles :

- `double Math.random()` : retourne un nombre aléatoire comprise entre 0 et 1
- `double sin = Math.sin(120);`
- `double cos = Math.cos(120);`
- `double tan = Math.tan(120)` : retourne le tangente
- `double abs = Math.abs(-120.25)` : retourne la valeur absolu de x
- `double exp = Math.pow(d, 5);` retourne un nombre exposant n
- `double rac= Math.sqrt(x)` : Retourne la racine carrée
- `Math.max(x, y)` : Retourne le maximum entre x et y.
- `Math.min(x, y)` : Retourne le minimum entre x et y.
- `Math.round(x)` : Arrondit x à l'entier le plus proche.
- `Math.ceil(x)` : Arrondit x à l'entier supérieur.
- `Math.floor(x)` : Arrondit x à l'entier inférieur.
- `Math.cbrt(x)` : Retourne la racine cubique de x.

## 5.2. METHODE PERSONNALISE

### 5.2.1. Création

Pour créer de nouvelle méthode il nous sera logique de partir sur base des méthodes natives comme « `nextLine()`, `main()` et `Math.sqrt()` ».

- `Math.sqrt(25) = 5` calcule la racine carrée de 25 ;
- `nextLine()` donne la possibilité à l'utilisateur de faire la saisi ;
- etc.

Créer sa propre méthode en Java est une compétence essentielle pour organiser et réutiliser du code de manière efficace. Voici comment vous pouvez définir votre propre méthode en Java :

- Utilisez le mot-clé *public* pour rendre la méthode accessible depuis d'autres classes.
- *Static* : signifie que la méthode appartient à la Classe principale
- le mot-clé *void* signifie que cette méthode n'a pas une Valeur de retour.
- Définissez le nom de la méthode et ses paramètres entre parenthèses.
- Le corps de la méthode est délimité par des accolades {}.

```
public static void afficherEnTete() {
    System.out.println("Bienvenue à GIRMELAB - Vente d'ordinateurs portables");
    System.out.println("Téléphone : 123456789");
    System.out.println("Adresse : Loma, Mbanza-Ngungu");
    System.out.println("=====");
}
```

### 5.2.2. Appeler une méthode

Pour appeler une méthode en Java, écrivez le nom de la méthode suivi de deux parenthèses () et un point-virgule ;

```
public static void main(String[] args) {
    afficherEnTete();
}
```

```
Bienvenue ♦ GIRMELAB - Vente d'ordinateurs portables
T♦l♦phone : 123456789
Adresse : Loma, Mbanza-Ngungu
=====
```

Une méthode peut être appelée plusieurs fois

```
public class Essaie {
    static void maMt() {
        System.out.println("I just got executed!");
    }
    public static void main(String[] args) {
```

```
maMt();
maMt();
maMt();
```

### 5.2.3. Paramètres de la méthode Java

L'information peut être transmise aux méthodes sous forme de paramètre. Les paramètres agissent comme des variables à l'intérieur de la méthode.

Les paramètres sont spécifiés après le nom de la méthode, entre parenthèses. Vous pouvez ajouter autant de paramètres que vous voulez, il suffit de les séparer avec une virgule.

```
public class Main {
    static void maMt(String prenom) {
        System.out.println(fname + " Mukoko");
    }

    public static void main(String[] args) {
        maMt ("Lucia");
        maMt ("bethel");
        maMt ("girmela");
    }
}
```

Note : Lorsqu'un **paramètre** est transmis à la méthode, il est appelé **un argument**. Si votre méthode à plusieurs paramètres, l'appel de méthode doit avoir le même nombre d'arguments qu'il y a de paramètres, et les arguments doivent être passés dans le même ordre.

```
public static double MontantTotal(int quantite, double prixUnitaire) {
    double montantTotal = quantite * prixUnitaire;
    return montantTotal;
}
```

Cette méthode retourne une valeur.

```
System.out.println( " Le prix total est " + MontantTotal(12,5));
```

```
Le prix total est 60.0
```

Nous allons encore entrer en profondeur sur ce sujet dans le prochain chapitre.

```

Bienvenue ♦ GIRME LAB - Vente d'ordinateurs portables
T♦l♦phone : 123456789
Adresse : Loma, Mbanza-Ngungu
=====
Le prix total est 60.0
BUILD SUCCESSFUL (total time: 0 seconds)

```

Figure 8: Résultat du programme

#### 5.2.4. Une méthode avec Si... Sinon

Il est courant d'utiliser des instructions à l'intérieur des méthodes :if...else

```

public class Main {
    static void Age(int age) {
        if (age >= 18) {
            System.out.println("l'accès est payant ! ");
        } else {
            System.out.println("l'accès est gratuit!");
        }
    }
    public static void main(String[] args) {
        Age(20);
    }
}

```

Plusieurs méthodes peuvent porter le même nom avec différentes Paramètres ou différents types de données et dans ce cas on parlera de la *surcharge de méthode*.

#### 5.2.5. Récursion

La récursivité est la technique qui consiste à faire appel à une fonction elle-même. Cette technique offre un moyen décomposer des problèmes complexes en problèmes plus simples et plus faciles à résoudre.

La récursivité peut être un peu difficile à comprendre. La meilleure façon de comprendre comment ça fonctionne, c'est d'expérimenter.

```

public static int sum(int k) {
    if (k > 0) {
        return k + sum(k - 1);
    } else {
        return 0;    } }

public static void main(String[] args) {
    int result = sum(10);
    System.out.println(result);
}

```

```

public static int puissance(int base, int exposant) {
    if (exposant == 0) {
        return 1;
    } else {
        return base * puissance(base, exposant - 1);    }
}

```

### Condition d'arrêt d'une méthode récursive

Tout comme les boucles peuvent rencontrer le problème du bouclage infini (boucle infernale), les méthodes récursives peuvent rencontrer Le problème de la récursion infinie. La récursion infinie est lorsque la méthode ne cesse jamais d'appeler lui-même. Toute méthode récursive doit avoir une condition d'arrêt, qui est la condition où la méthode cesse de s'appeler elle-même. Dans l'exemple précédent, la condition d'arrêt est lorsque le paramètre(k) devient 0 et lorsque le paramètre(exposant) devient 0

## EXERCICES SUR LES METHODES PERSONNALISEES

1. Ecrire un programme java permettant à l'utilisateur de saisir un nombre et en suite qu'il calcul sa factoriel
2. Ecrire un programme java permettant à l'utilisateur de saisir un mots, nom... et quant suite le programme affiche son inverse

## **6. Programmation Orientée Objet (POO) en Java**

### **6.1. CONCEPTS FONDAMENTAUX**

La programmation procédurale consiste à écrire des procédures ou des méthodes qui fonctionnent sur les données, tandis que la programmation orientée objet concerne créer des objets contenant à la fois des données et des méthodes.

#### **6.1.1. Définition de la POO**

La Programmation Orientée Objet (POO) est un paradigme de programmation qui repose sur la modélisation de concepts du monde réel sous forme d'objets interagissant les uns avec les autres.

La programmation orientée objet présente plusieurs avantages :

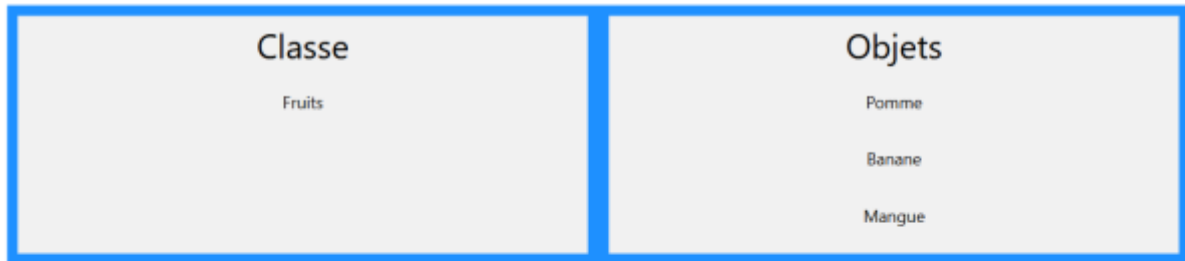
- La POO est plus rapide et facile à exécuter
- La POO fournit une structure claire pour les programmes
- La POO aide à garder le code Java SEC « Don't Repeat You » (Ne pas te répéter), et rend le code plus facile à maintenir, modifier et déboguer
- La POO permet de créer des objets entièrement réutilisables Applications avec moins de code et un temps de développement plus court

#### **Classes et objets**

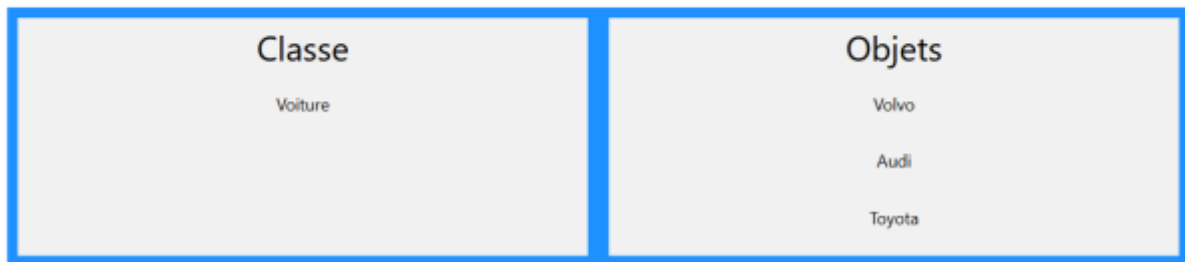
Une classe peut être comparée à un moule qui, lorsque nous le remplissons, nous donne un objet ayant la forme du moule ainsi que toutes ses caractéristiques.

Ou encore

Une classe est un modèle qui définit les attributs et comportements des objets et un objet est une instance spécifique d'une classe, représentant une entité concrète.



Un autre exemple :



## Structure de base

```
class ClasseMain{
  Attributs et méthodes
}
```

Voici la structure d'une classe jusqu'à maintenant ! Ah ! oui cette structure n'a pas changé, elle est la même. A partir de maintenant nous allons devoir créer d'autre CLASSE.

Nous allons créer la classe L2\_IG, Il ne peut y avoir qu'une seule méthode **main active** par projet ! Souvenez-vous que celle-ci est le point de départ de votre programme. Pour être tout à fait précis, plusieurs méthodes main peuvent cohabiter dans votre projet, mais une seule sera considérée comme le point de départ de votre programme !

```
Public class l2_IG{

}
```

Ici, notre classe L2\_IG est précédée du mot clé public. Vous devez savoir que lorsque nous créons une classe comme nous l'avons fait en pratique, netbeans nous ajoute automatiquement ce mot clé, qui correspond à la portée de la classe.

Un objet est une entité concrète qui représente une occurrence particulière d'une classe. Chaque objet possède ses propres attributs (état) et méthodes (comportement) qui le distinguent des autres objets.

### **Instance d'une Classe**

Pour créer un objet en Java, vous utilisez le mot-clé `new` suivi du nom de la classe et d'éventuels arguments de constructeur. Par exemple, `Personne personnel = new Personne ("Alice", 30)` ; crée un objet de type `Personne`.

### **Attributs**

Les attributs représentent les données ou les caractéristiques d'un objet. Chaque objet créé à partir d'une classe possède ses propres valeurs d'attributs qui définissent son état.

Les attributs sont déclarés dans une classe en spécifiant leur type de données et leur nom. Par exemple, `private String nom ;` déclare un attribut de type chaîne de caractères nommé "nom".

En programmation, la portée détermine qui peut faire appel à une classe, une méthode ou une variable. Vous avez déjà rencontré la portée publique : cela signifie que tout le monde peut faire appel à l'élément. Nous allons ici utiliser une autre portée : *private*. Elle signifie que notre méthode ne pourra être appelée que depuis l'intérieur de la classe dans laquelle elle se trouve ! Les méthodes déclarées *private* correspondent souvent à des mécanismes internes à une classe que les développeurs souhaitent « cacher » ou simplement ne pas rendre accessibles de l'extérieur de la classe. . .

### **Méthodes**

Les méthodes sont des blocs de code qui définissent les actions qu'un objet peut effectuer. Par exemple, dans une classe `Voiture`, les méthodes pourraient inclure `démarrer ()`, `accélérer ()`, `freiner ()`, etc.

Les méthodes, également appelées fonctions ou comportements, définissent le comportement d'un objet.

Note :

- Private : Accessible uniquement à l'intérieur de la classe.
- Public : Accessible depuis n'importe où.
- Protected : Accessible dans la classe et ses sous-classes.
- Sans spécificateur : Visibilité par défaut dans le package.

## Les constructeurs

Un constructeur est une méthode spéciale dans une classe Java qui est appelée lorsqu'un objet de cette classe est créé à l'aide de l'opérateur *new*. Son rôle est d'initialiser les attributs de l'objet.

Un constructeur est caractérisé par :

- Le nom d'un constructeur est le même que le nom de la classe.
- Un constructeur n'a pas de type de retour défini, même pas void.
- Les constructeurs peuvent être surchargés, ce qui signifie qu'une classe peut avoir plusieurs constructeurs avec des paramètres différents.

```
public class Personne {  
    private String nom;  
    private int age;  
    // Constructeur avec paramètres  
    public Personne(String nom, int age) {  
        this.nom = nom;  
        this.age = age;  
    }  
    public static void main ( String[] args) {  
        Personne personne1 = new Personne ("Alice", 30);  
    }  
}
```

```

package opérateurs;

public class Voiture {
    private String marque;
    private String modele;
    private int anneeFabrication;

    public Voiture() {
        this.marque = "Non spécifiée";
        this.modele = "Non spécifié";
        this.anneeFabrication = 0;
    }

    // Nous voulons surcharger la méthode (Constructeur)
    public Voiture(String marque, String modele, int anneeFabrication) {
        this.marque = marque;
        this.modele = modele;
        this.anneeFabrication = anneeFabrication;
    }

    public void afficherDetails() {
        System.out.println("Marque : " + marque);
        System.out.println("Modèle : " + modele);
        System.out.println("Année de fabrication : " + anneeFabrication);
    }

    public static void main(String[] args) {
        Voiture maVoiture = new Voiture();
        Voiture tavoiture= new Voiture("TOYOTA","PRADO",1999);
        maVoiture.afficherDetails();
        tavoiture.afficherDetails();
    }
}

```

Figure 9: constructeur par défaut et un constructeur surchargé

## Accesseurs(getters) et mutateurs(setters)

Un accesseur est une méthode qui va nous permettre d'accéder aux variables de nos objets en lecture, et un mutateur nous permettra d'en faire de même en écriture ! Grâce

aux accesseurs, vous pourrez afficher les variables de vos objets, et grâce aux mutateurs, vous pourrez les modifier :

Nos accesseurs sont bien des méthodes, et elles sont public pour que vous puissiez y accéder depuis une autre classe que celle-ci : depuis le « main », par exemple. Les accesseurs sont du même type que la variable qu'ils doivent retourner. Les mutateurs sont, cependant, de type void. Ce mot clé ne signifie « rien » ; en effet, ces méthodes ne retournent aucune valeur, elles se contentent de les mettre à jour.

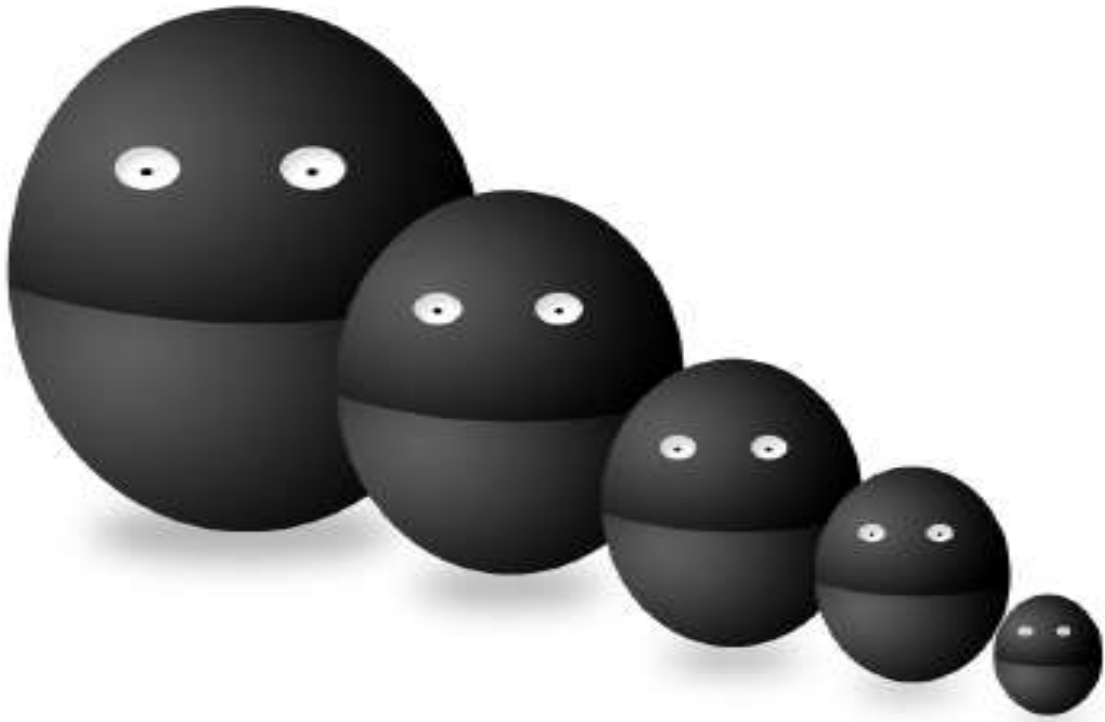
## **Encapsulation**

Encapsulation est l'effet de caché ou protégé les variables d'une classe en lui donnant une portée *protected*. En effet, depuis l'extérieur de la classe, elles ne sont accessibles que via les accesseurs et mutateurs que nous avons définis. C'est le principe d'encapsulation !

En fait, lorsqu'on procède de la sorte, on s'assure que le fonctionnement interne à l'objet est intègre, car toute modification d'une donnée de l'objet est maîtrisée. Nous avons développé des méthodes qui s'assurent qu'on ne modifie pas n'importe comment les variables.

## **6.2. HERITAGE**

La notion d'héritage en programmation est différente de celle que vous connaissez, bien qu'elle en soit tout de même proche. C'est l'un des fondements de la programmation orientée objet !



*Figure 10: Notion sur l'héritage*

La notion d'héritage est l'un des fondements de la programmation orientée objet. Grâce à elle, nous pourrions créer des classes héritées de nos classes mères (classe de base). Nous pourrions créer autant de classes dérivées, par rapport à notre classe de base, que nous le souhaitons. De plus, nous pourrions nous servir d'une classe dérivée comme d'une classe de base pour élaborer encore une autre classe dérivée (classe fille ou classe héritée).

### **6.2.1. Définition**

L'héritage permet à une classe de reprendre les attributs et les méthodes d'une autre classe, en établissant une relation "est un" entre les classes. Cela favorise la réutilisation du code et la hiérarchisation des classes.

La classe parente est celle dont les caractéristiques sont héritées. Elle peut contenir des attributs et des méthodes générales.

La classe fille hérite des attributs et des méthodes de la classe parente et peut également définir ses propres attributs et méthodes spécifiques.

### 6.2.2. Utilisation de l'héritage

L'héritage crée une relation "**est un**", ce qui signifie qu'une classe fille est une version spécialisée de sa classe parente. Par exemple, un chien **est un** animal.

L'agrégation ou la composition crée une relation "**a un**", indiquant qu'un objet contient un autre objet.

On peut redéfinir une méthode héritée, c'est-à-dire qu'on peut changer tout son code.

On peut utiliser le comportement d'une classe mère par le biais du mot clé `super`.

#### Mots-clés en Héritage :

- `extends` : Utilisé pour déclarer l'héritage d'une classe parente.
- `super` : Référence à la classe parente, utilisée pour appeler les membres de la classe parente.
- `@Override` : Annotation pour indiquer que la méthode redéfinit une méthode de la classe parente.

#### Exemple

Le programme modélise des entités de type `Personne`, `Etudiant` et `Employe`. Chaque `Personne` peut parler, chaque `Etudiant` peut étudier et passer des examens, tandis que chaque `Employe` peut travailler et évaluer sa performance au travail.

## 7. Interfaces Graphiques en Java

Dans ce chapitre, nous aborderons les interfaces graphiques (on parle aussi d'IHM pour Interfaces Homme Machine ou de GUI pour Graphical User Interfaces) et, par extension, la programmation événementielle. Par là, vous devez comprendre que votre programme ne réagira plus à des saisies au clavier mais à des événements provenant d'un composant graphique : un bouton, une liste, un menu. . .

Le langage Java propose différentes bibliothèques pour programmer des IHM, mais dans cet ouvrage, nous utiliserons essentiellement les packages `javax.swing` et `java.awt` présents d'office dans Java.

Ce chapitre vous permettra d'apprendre à utiliser l'objet `JFrame`, présent dans le package `java.swing`. Vous serez alors à même de créer une fenêtre, de définir sa taille, etc. Le fonctionnement de base des IHM vous sera également présenté et vous apprendrez qu'en réalité, une fenêtre n'est qu'une multitude de composants posés les uns sur les autres et que chacun possède un rôle qui lui est propre.

Swing est une bibliothèque graphique incluse dans la plate-forme Java qui offre un ensemble complet de composants graphiques (widgets) pour la création d'interfaces utilisateur (UI) dans les applications Java.

### 7.1. COMPOSANT DE SWING

Swing, la magnifique bibliothèque graphique Java qui regorge de composants impressionnants pour créer des interfaces utilisateur élégantes et interactives ! Laissez-moi vous présenter Swing et quelques-uns de ses composants les plus couramment utilisés :

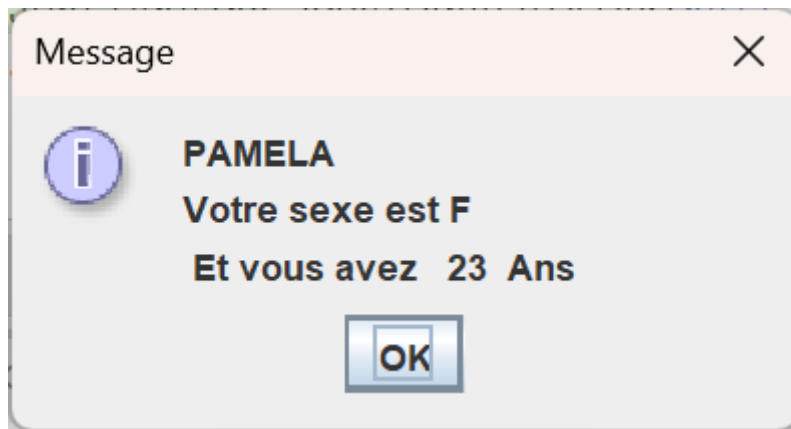
#### 7.1.1. Utilisation de la classe `JOptionPane`

Java, nous propose utilisation des interfaces prédéfinie qu'on appelle communément boîtes de dialogue. La classe `JOptionPane` du package « `javax.swing` », nous permet de d'avoir les boîtes de dialogue.

Les boîtes de dialogue `JOptionPane` en Java sont des composants graphiques qui permettent d'afficher des messages, des demandes d'entrées de l'utilisateur et des options de sélection dans une interface graphique. Voici quelques points clés à retenir sur l'utilisation de `JOptionPane` :

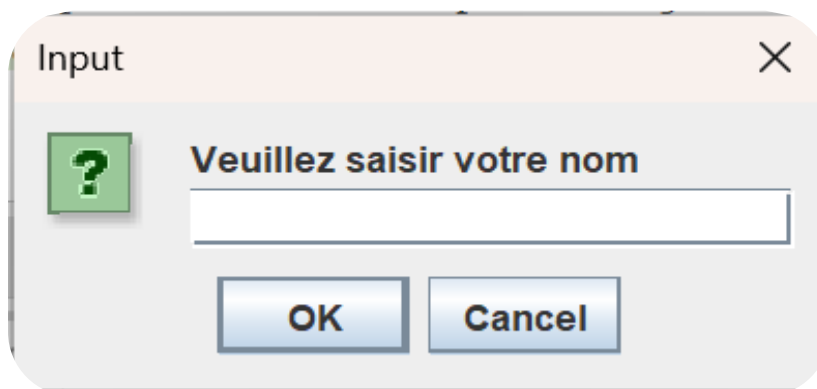
### Affichage de messages

Vous pouvez utiliser `JOptionPane` pour afficher des messages informatifs, d'avertissement, d'erreur ou de confirmation à l'utilisateur. Les méthodes les plus couramment utilisées sont `showMessageDialog()`, `showConfirmDialog()` et `showOptionDialog()`.



### Demande d'entrées

`JOptionPane` permet également de demander des entrées à l'utilisateur, comme des chaînes de caractères, des entiers, des doubles, etc. Les méthodes telles que `showInputDialog()` sont utiles pour cela mais, toutes valeurs saisies par l'util



```
String nom=JOptionPane.showInputDialog(null, "Veuillez saisir votre nom");
char Sexe=JOptionPane.showInputDialog(null, "Veuillez saisir sexe").charAt(0);
int age=Integer.parseInt(JOptionPane.showInputDialog(null, "Saisir votre Age"));
JOptionPane.showMessageDialog(null, nom+" \n"+"Votre sexe est "+Sexe+" \nEst vous avez "+age+" Ans");
```

## Types de messages

Les boîtes de dialogues JOptionPane peuvent afficher différents types de messages en utilisant les constantes de la classe JOptionPane telles que INFORMATION\_MESSAGE, WARNING\_MESSAGE, ERROR\_MESSAGE, QUESTION\_MESSAGE, etc.

## Boutons personnalisés

Vous pouvez personnaliser les boutons affichés dans une boîte de dialogue JOptionPane en spécifiant les options possibles à l'aide des constantes de la classe JOptionPane, comme YES\_NO\_OPTION, OK\_CANCEL\_OPTION, etc.

## Gestion des retours

Lorsque vous utilisez JOptionPane pour demander des entrées à l'utilisateur, assurez-vous de vérifier et de traiter correctement la valeur de retour pour prendre des décisions en conséquence dans votre programme. Une boîte de saisie renvoie une chaîne de caractère.

1. **JFrame** : Le JFrame est une fenêtre top-level (de premier plan) qui représente la fenêtre principale d'une application Swing. Il peut contenir d'autres composants tels que des boutons, des panneaux, des champs de texte, etc.
2. **JPanel** : Le JPanel est un conteneur léger qui peut être utilisé pour organiser et regrouper d'autres composants graphiques. Il est souvent utilisé pour structurer l'interface utilisateur en sections logiques.

3. **JButton** : Le JButton est un composant qui représente un bouton cliquable pouvant déclencher des actions lorsque l'utilisateur interagit avec lui, par exemple en cliquant dessus.
4. **JLabel** : Le JLabel est un composant utilisé pour afficher du texte ou une image statique à l'écran. Il est généralement utilisé pour afficher des informations descriptives dans l'interface utilisateur.
5. **TextField** : Le JTextField est un composant de champ de texte simple où l'utilisateur peut saisir du texte. Il est largement utilisé pour la saisie de données dans les applications GUI.
6. **TextArea** : Le JTextArea est un composant de zone de texte multi-lignes qui permet à l'utilisateur de saisir et afficher du texte sur plusieurs lignes.
7. **CheckBox** : Le JCheckBox est un composant de case à cocher qui permet à l'utilisateur de sélectionner une ou plusieurs options à la fois.
8. **ComboBox** : Le JComboBox est un composant de liste déroulante qui permet à l'utilisateur de sélectionner une option parmi une liste d'éléments.
9. **ScrollPane** : Le JScrollPane est un composant utilisé pour ajouter des barres de défilement à d'autres composants tels que JTextArea ou JTable lorsque le contenu dépasse la taille de la zone d'affichage.

### 7.1.2. L'objet JFrame

Un JFrame est une classe de la bibliothèque javax.swing de Java qui représente une fenêtre graphique avec des bordures, des boutons de fermeture, de réduction et de maximisation. C'est l'élément de base pour créer des interfaces graphiques dans les applications Java.

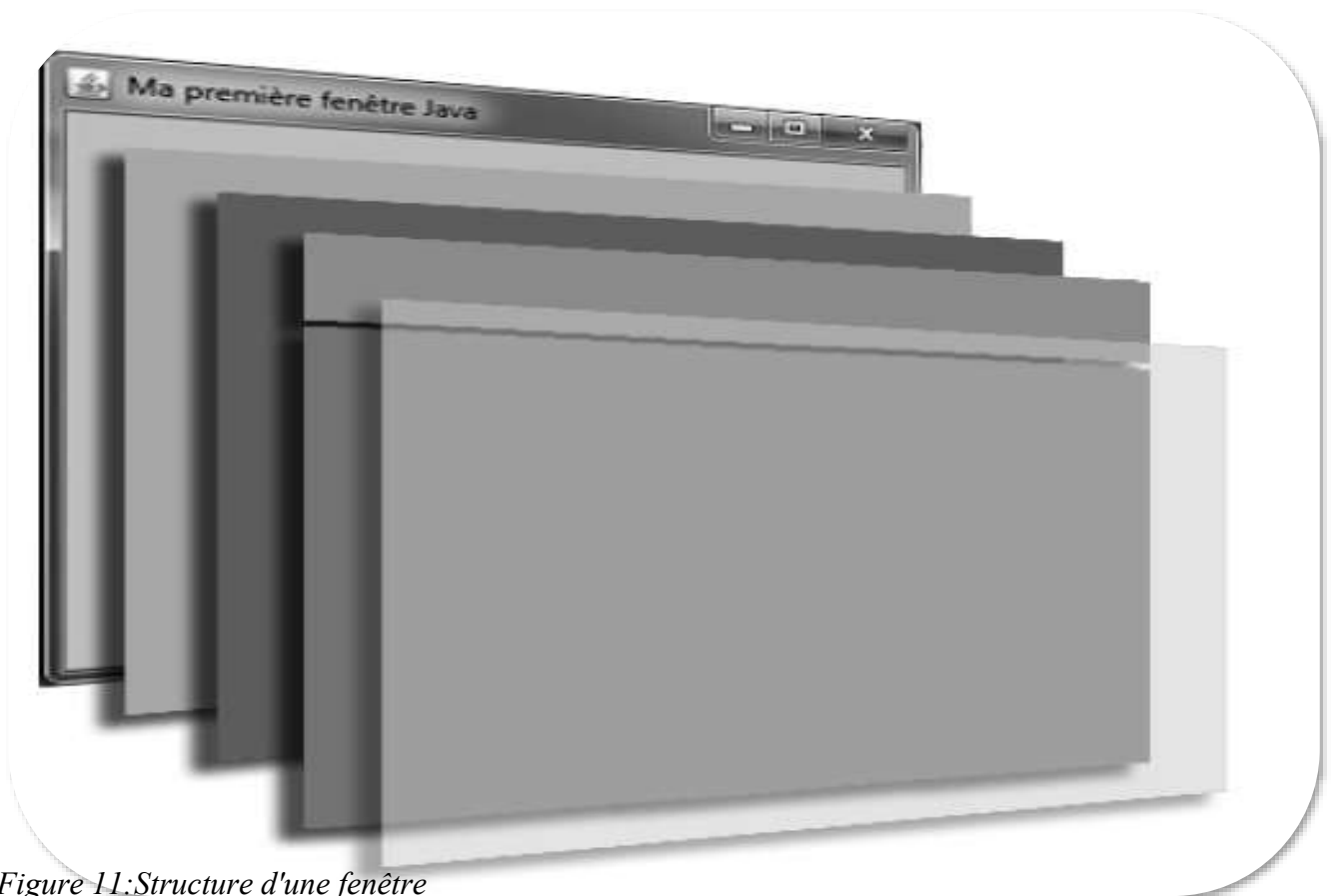


Figure 11: Structure d'une fenêtre

## 1. Création d'une fenêtre de base

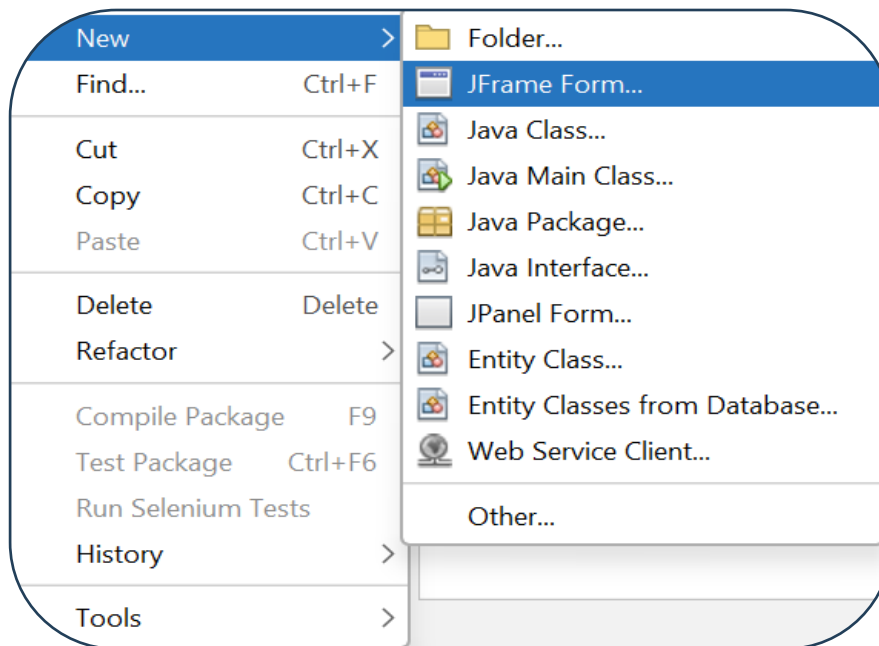
Nous allons évoluer en WYSIWIG mais en s'appuyant sur certaines notions de la programmation orienter. Après avoir créé le projet :

- Clic droit sur le package qui porte le même nom que le projet



Figure 12: structure d'un projet

- Pointez sur new
- Cliquez sur JFram form



- Donnez le nom de l'interface
- Cliquez ensuite sur FINISH

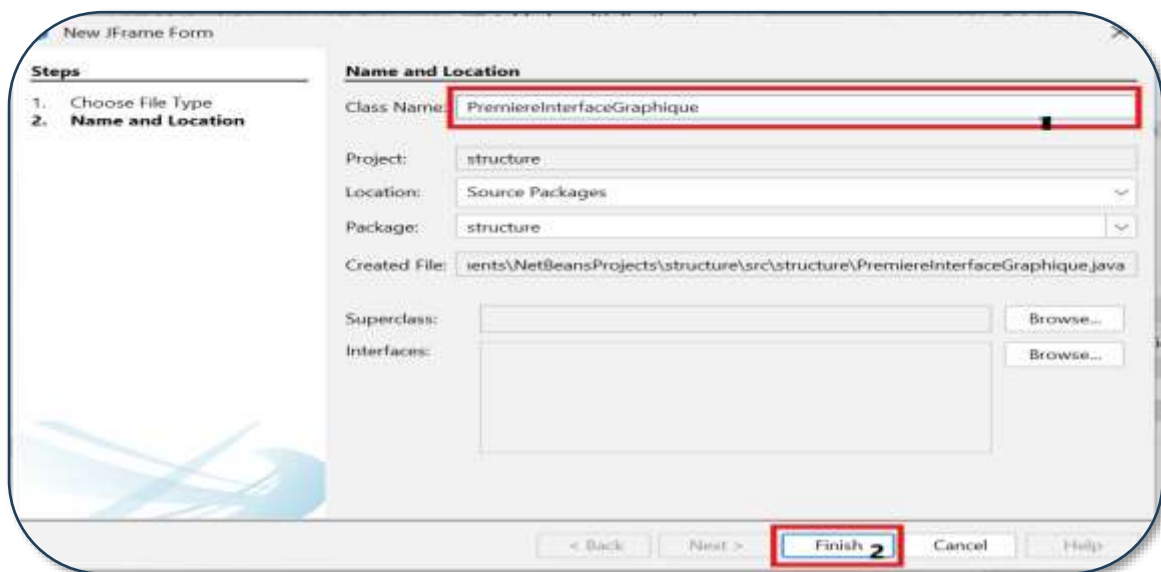


Figure 13:Création d'une interface graphique

- Et voilà l'interface

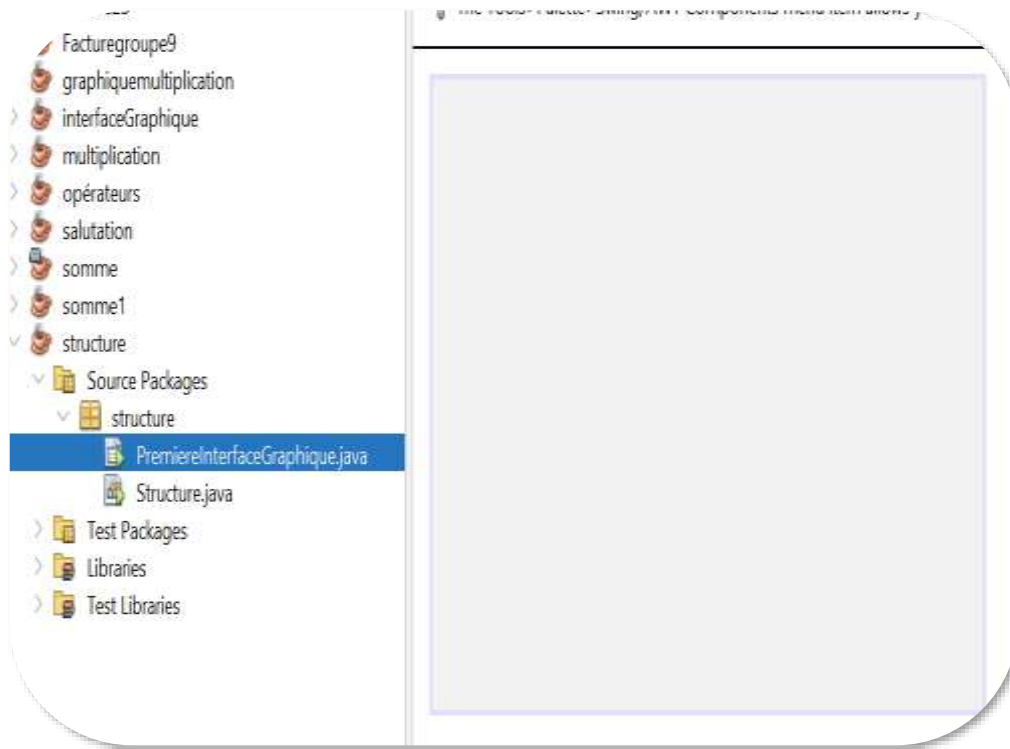


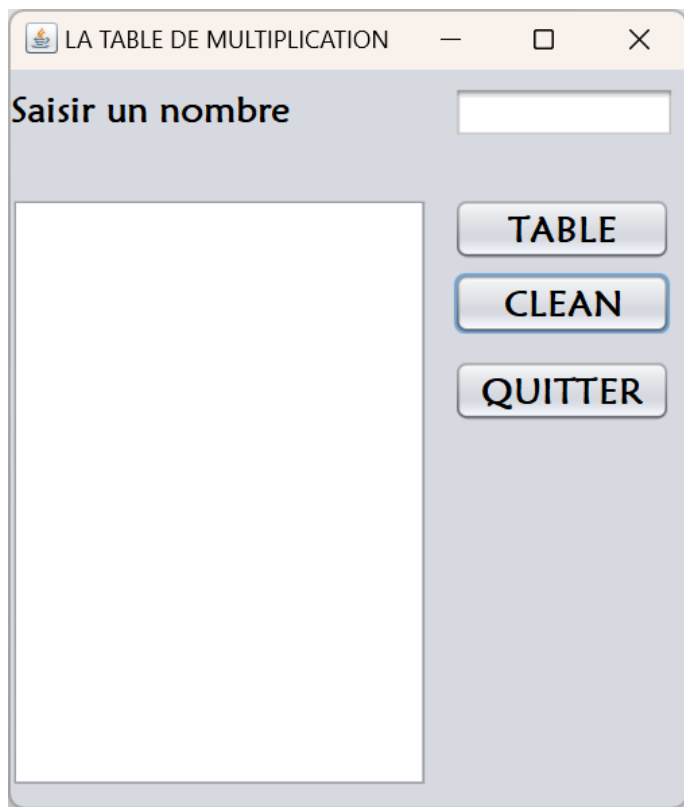
Figure 14: interface graphique d'un JFrame

Vous pouvez ajouter divers composants graphiques tels que des boutons, des libellés, des champs de texte, etc., à un JFrame en utilisant des conteneurs tels que JPanel ou directement sur le JFrame lui-même.

## 2. Quelques propriétés de JFrame form

- **Taille et position** : Vous pouvez définir la taille et la position d'un JFrame en utilisant les méthodes `setSize (int width, int height)` et `setLocation (int x, int y)`.
- **Visibilité** : Pour rendre un JFrame visible à l'écran, utilisez la méthode `setVisible(true)`. Vous pouvez également masquer la fenêtre en définissant la visibilité sur `false`.
- **Fermeture de fenêtre** : Vous pouvez définir le comportement de fermeture de la fenêtre en utilisant `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)` pour fermer l'application lorsque la fenêtre est fermée.
- **Gestion des événements** : Vous pouvez ajouter des écouteurs d'événements pour gérer les interactions utilisateur, tels que les clics de souris ou les saisies clavier.

champs de texte ('JTextField'), labels ('JLabel')



- Menus et barres d'outils
- 4.3. Gestion des Événements
- Écouteurs d'événements ('ActionListener', 'MouseListener')
- Gestion des interactions utilisateur
- 4.4. Disposition des Composants
- Gestion des layouts ('FlowLayout', 'BorderLayout', 'GridLayout')
- Personnalisation de l'interface
- 4.5. Introduction à JavaFX
- Différences entre Swing et JavaFX
- Création d'une application JavaFX de base

## 2.5. GESTION DES ERREURS

- Exceptions en Java
- Bloc 'try-catch' et gestion des exceptions
- Création d'exceptions personnalisées

## 5. Projets Pratiques

- 5.1. Projet 1 : Calculatrice
- Application console et interface graphique
- Utilisation de la POO pour modéliser les opérations
- 5.2. Projet 2 : Gestionnaire de Contacts
- Modélisation des contacts avec des classes
- Création d'une interface graphique pour ajouter et afficher des contacts
- 5.3. Projet 3 : Application de Gestion de Tâches
- Utilisation de listes pour gérer les tâches
- Interface graphique pour l'ajout, la modification et la suppression de tâches

## 6. Conclusion

- 6.1. Récapitulatif des Concepts
- Importance de la POO et des interfaces graphiques en Java
- Ressources pour approfondir les connaissances
- 6.2. Prochaines Étapes
- Conseils pour continuer l'apprentissage (livres, cours en ligne)
- Introduction à des concepts avancés (multithreading, bases de données)

## Annexe

- Exemples de Code
- Exemples pratiques pour chaque section clé
- Ressources Utiles
- Liens vers la documentation Java, tutoriels vidéo, forums d'entraide